



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

Техническое задание

на разработку «Распределенной информационной системы (SherToKras)
бронирования авиабилетов из аэропорта Шереметьево в аэропорт города
Красноярск»

Подп. и дата	
Инв.№ дубл.	
Взаим.инв.№	
Подп. и дата	
Инв.№ подл.	

Студент ИУ7-22М
(группа)

(Подпись, дата)

Агарков М. В.
(Фамилия И.О.)

Преподаватель

(Подпись, дата)

Романова Т. Н.
(Фамилия И.О.)

СОДЕРЖАНИЕ

Глоссарий	4
Введение	5
Основания для разработки	6
Назначение разработки	6
Существующие аналоги	6
Описание системы	7
Требования к функциональным характеристикам	8
Функциональные требования к portalу с точки зрения пользователя	8
Входные данные	10
Выходные параметры	12
Топология системы	14
Требования по реализации со стороны заказчика	19
Функциональные требования по подсистемам	20
Сценарий взаимодействия фронтенда и бекендов	22
Сценарий взаимодействия бекендов	22
Пользовательский интерфейс	30
Требования к дизайну	31
Требования к составу и параметрам технических средств	36
Требования к надежности	36
Требования к документации	36
Проектирование системы	37
Концептуальный дизайн	37
Концептуальная модель системы в нотации IDEF0	37
Сценарии функционирования системы	38
Диаграммы прецедентов	42
Спецификация сценария «регистрация пользователя»	42
Спецификация сценария «авторизация пользователя»	44
Спецификация сценария «покупка билета»	45
Логический дизайн	50
Диаграммы классов	50
Диаграмма действий	63
Диаграмма последовательности действий	64

Диаграмма потоков данных	64
Высокоуровневый дизайн пользовательского интерфейса	66
Физический дизайн	69
Выбор системы развертывания компонентов распределенной системы	69
Выбор операционной системы	71
Выбор СУБД	71
Обеспечение надежности портала	73

Глоссарий

1. Узел системы – региональный сервер, содержащий данные авторов и читателей указанного региона;
2. Валидация – проверка данных на соответствие заданным условиям и ограничениям;
3. REST – архитектурный стиль взаимодействия компонентов распределённого приложения в сети;
4. Медиана времени отклика – среднее время предоставления данных пользователю;
5. Латентность географического положения – увеличение времени отклика приложения, обуславливаемое географическим положением элементов системы или пользователя.
6. Авторизованный пользователь – пользователь, успешно прошедший проверку подлинности (аутентификацию) и получивший доступ к определённым ресурсам или функциям системы в соответствии со своими правами.
7. Неавторизованный пользователь – пользователь, не прошедший проверку подлинности или не обладающий необходимыми правами доступа, в результате чего его возможности в системе ограничены или отсутствуют вовсе.
8. Администратор – пользователь с расширенными полномочиями, который управляет настройками системы, пользователями, правами доступа и следит за корректной работой программного обеспечения.
9. Аутентификация – процесс проверки подлинности пользователя или устройства.
10. Авторизация – это процесс проверки прав доступа.
11. OpenID Connect – это протокол аутентификации и авторизации, который строится на основе протокола OAuth 2.0.

12. Identity Provider – это сервис, который управляет аутентификацией и предоставляет информацию о пользователе в рамках системы аутентификации и авторизации;
13. JSON – это популярный формат текстовых данных, который используется для обмена данными в современных веб - и мобильных приложениях;
14. JSON Web Token (JWT) – объект, состоящий из трех частей: заголовка (header), полезной нагрузки (payload) и подписи; является открытым стандартом (RFC 7519) для создания токенов доступа, основанный на формате JSON;
15. Kubernetes – открытое программное обеспечение для оркестровки контейнеризированных приложений, автоматизации их развёртывания, масштабирования и координации в условиях кластера;
16. Docker – Представляет собой набор продуктов «платформа как услуга» (PaaS), которые используют виртуализацию на уровне ОС для доставки программного обеспечения в пакетах, называемых контейнерами.

Введение

В условиях активного развития авиационной инфраструктуры и роста пассажиропотока между регионами России возрастает необходимость в эффективных и отказоустойчивых системах бронирования авиабилетов. Особенно актуальной становится задача автоматизации бронирования авиаперелётов из крупнейших транспортных узлов, таких как аэропорт Шереметьево, в города Сибири, где требуется учёт множества направлений, рейсов и перевозчиков.

Настоящий проект представляет собой техническое задание на разработку распределённой информационной системы бронирования авиабилетов (SherToKras), предназначенной для обеспечения удобного, быстрого и безопасного бронирования авиаперелётов из московского аэропорта Шереметьево в аэропорт города Красноярск. Система будет учитывать распределённый характер обработки данных, обеспечивать устойчивость к нагрузкам и отказам, а также поддерживать интеграцию с внешними сервисами авиакомпаний и платёжных систем.

Техническое задание выполнено на основе ГОСТ 19.201–78 «ЕСПД. Техническое задание. Требования к содержанию и оформлению».

Основания для разработки

Разработка ведется в рамках выполнения лабораторных работ по курсу «Методология программной инженерии» на кафедре «Программное обеспечение ЭВМ и информационные технологии» факультета «Информатика и системы управления» МГТУ им. Н. Э. Баумана.

Назначение разработки

Главное назначение разрабатываемого портала — предоставить пользователю удобный способ бронирования авиабилетов из одного из самых популярных московских аэропортов — Шереметьево — в город Красноярск. Аэропорт Шереметьево выбран в качестве отправной точки, поскольку на сегодняшний день именно из него совершается большинство рейсов из Москвы в Красноярск.

Система будет обеспечивать поиск, онлайн-бронирование, управление текущими заявками и использование бонусных программ независимо от конкретной авиакомпании.

Дополнительно предполагается, что сервис станет инструментом стимулирования внутреннего туризма, облегчая посещение города Красноярск за счёт скорости и простоты оформления авиабилетов.

Существующие аналоги

Среди аналогов разрабатываемого проекта можно выделить следующие:

- **heathrow.com/flights**: Портал является официальным сайтом лондонского аэропорта Хитроу и предоставляет информацию о расписании вылетов и прилётов. Хотя на сайте доступна подписка на уведомления о статусе рейсов и возможность перехода к бронированию через партнёрские платформы, полноценного модуля для оформления билетов, личного кабинета и функций возврата билетов не предусмотрено.
- **moscowairports.ru**: Портал объединяет информацию о московских аэропортах и предлагает сервис для поиска авиабилетов с вылетом

только из Шереметьево, Домодедово и Внуково. Несмотря на возможность найти и сравнить предложения, само бронирование происходит через внешние агрегаторы. На сайте отсутствуют функции управления бронированиями, возврата билетов и программы лояльности.

- **spotaero.ru**: Сервис представляет собой российский веб-ресурс, специализирующийся на продаже билетов на региональные и чартерные рейсы, в том числе с вылетами из аэропортов Сибири. Он ориентирован на работу с ограниченными маршрутами и туроператорами, однако обладает ограниченной функциональностью: отсутствует личный кабинет пользователя, нет встроенного механизма возврата билетов и системы лояльности.

Изучив плюсы и минусы существующих решений, можно выделить следующие преимущества, которыми должен обладать разрабатываемый проект:

1. собственная система лояльности: все приведённые выше сервисы являются лишь агрегаторами авиабилетов, но не предлагают систему лояльности;
2. просмотр активных бронирований: приведённые сервисы не имеют личного кабинета пользователя, а потому не предоставляют возможности просмотра купленных билетов;
3. возврат забронированных билетов: приведённые сервисы не имеют личного кабинета пользователя, а потому не предоставляют возможности возврата купленных билетов.

Описание системы

Разрабатываемый сервис представляет собой веб-систему для поиска и бронирования авиабилетов по маршруту «Шереметьево (Москва) — Красноярск». Для неавторизованных пользователей доступен только просмотр списка доступных рейсов с датами, временем вылета и базовыми тарифами. Чтобы оформить заказ и приобрести билет, пользователь должен пройти регистрацию, указав персональные данные (ФИО, адрес электронной почты, телефон и пароль). После авторизации появляется доступ к расширенным

функциям: выбор мест, оформление бронирования, привязка платёжного способа, управление личным кабинетом и просмотр истории своих перелётов.

Требования к функциональным характеристикам

1. Разрабатываемое программное обеспечение должно обеспечивать функционирование системы в режиме 24/7/365 со среднегодовым временем доступности не менее 99.9%.
2. Необходимо поддерживать возможность добавления нового узла во время работы системы без перезапуска.
3. Каждый узел должен автоматически восстанавливаться после сбоя в течение 15 минут.
4. Должна быть реализована деградация функциональности в случае отказа некритических источников.
5. По результатам работы модуля сбора статистики медиана времени отклика системы на запросы пользователя на получение информации не должна превышать 3 секунд без учета латентности географического расположения узла;
6. По результатам работы модуля сбора статистики медиана времени отклика системы на запросы, добавляющие или изменяющие информацию на портале не должна превышать 5 секунд без учета латентности географического расположения узла.

Функциональные требования к portalу с точки зрения пользователя

Portal должен обеспечивать реализацию следующих функций:

1. Регистрация и авторизация пользователей с валидацией вводимых данных как через интерфейс приложения, так и через популярные социальные сети.
2. Аутентификация пользователей.
3. Ролевая модель пользователей. Выделяются следующие роли:

- авторизованный пользователь;
- неавторизованный пользователь;
- администратор.

4. **неавторизованный** пользователь имеет следующий набор функций:

- поиск билетов;

5. **авторизованный пользователь** имеет следующий набор функций:

- поиск билетов;
- покупка билета, в том числе с использованием бонусных баллов;
- просмотр купленных билетов;
- возврат билета;
- просмотр истории начисления и списания бонусных баллов;
- просмотр остатка на счёте бонусных баллов.

6. **администратор** имеет следующий набор функций:

- функции пользователя
- просмотр статистики по сайту

Входные данные

Сущность	Входные данные
Регистрация пользователя	1. Фамилия, имя — не более 256 символов; Разрешенные символы для фамилии и имени: — Буквы русского и латинского алфавита (А-Я, а-я, А-Z, a-z) — Пробелы, дефис (-) Запрещенные: — Цифры — Спецсимволы: (! @ # \$ % ^ & * () _ + = [] { } / ? \ ~) 2. Логин — не более 256 символов; 3. Пароль — не более 128 символов; Разрешённые символы для логина и пароля: — Буквы латинского алфавита (А-Z, а-z) — Цифры (0-9) — Нижнее подчёркивание (_) — Точка (.), тире (-) Запрещены: — Пробелы — Русские буквы — Спецсимволы (! @ # * () { } [])

Продолжение таблицы 1

Сущность	Входные данные
	4. Роль (пользователь / администратор); 5. Телефон в формате +7XXXXXXXXXX; 6. E-mail в формате *@*. *.
Аутентификация пользователя	1. Логин; 2. Пароль.
Поиск рейсов	1. Аэропорт вылета и назначения (Шереметьево, Красноярск); 2. Дата вылета (ДД.ММ.ГГГГ); 3. Количество пассажиров; 4. Класс обслуживания (эконом, комфорт, бизнес, первый).
Бронирование билетов	1. Идентификатор рейса; 2. Данные пассажиров (ФИО, дата рождения, реквизиты документа); 3. Способ оплаты; 4. Email для подтверждения.
Возврат билета	1. Идентификатор брони; 2. Причина возврата (необязательно); 3. Дата возврата (ДД.ММ.ГГГГ).

Таблица 1 - Входные данные системы бронирования авиабилетов

Выходные параметры

Сущность	Входные данные
Неавторизованный пользователь	1. Поиск рейсов: — аэропорт вылета и назначения — дата вылета и возврата (если взят обратный билет) — найденные рейсы с краткой информацией (время, перевозчик, длительность, цена). 2. Детали рейса: — маршрут — время — класс обслуживания — стоимость
Авторизованный пользователь	1. Профиль пользователя: — имя — фамилия — e-mail — дата рождения — бонусный счёт 2. Мои бронирования:

Продолжение таблицы 2

Сущность	Входные данные
	— список рейсов — даты — направления — статусы — кнопка отмены/возврата. 3. Бонусная программа: — баланс бонусов — история начислений и списаний. 4. Поиск рейсов и оформление бронирования с применением бонусов и оплаты. 5. Информация о рейсе: детальные данные по выбранному перелёту.
Администратор	1. Список пользователей: — ФИО — e-mail — телефон — роль 2. Информация о бронированиях:

Продолжение таблицы 2

Сущность	Входные данные
	— рейс — пользователь — дата брони — статус оплаты — статус рейса 3. Детали пользователя: — личные данные — история действий — бонусный счёт 4. Панель статистики: — количество запросов — распределение по методам (GET/POST и т.д.), статусам, времени обработки 5. Просмотр всех рейсов: — идентификатор — маршрут — доступные места

Таблица 2 - Выходные параметры в зависимости от роли пользователя

Топология системы

Топология системы представлена на рисунке:

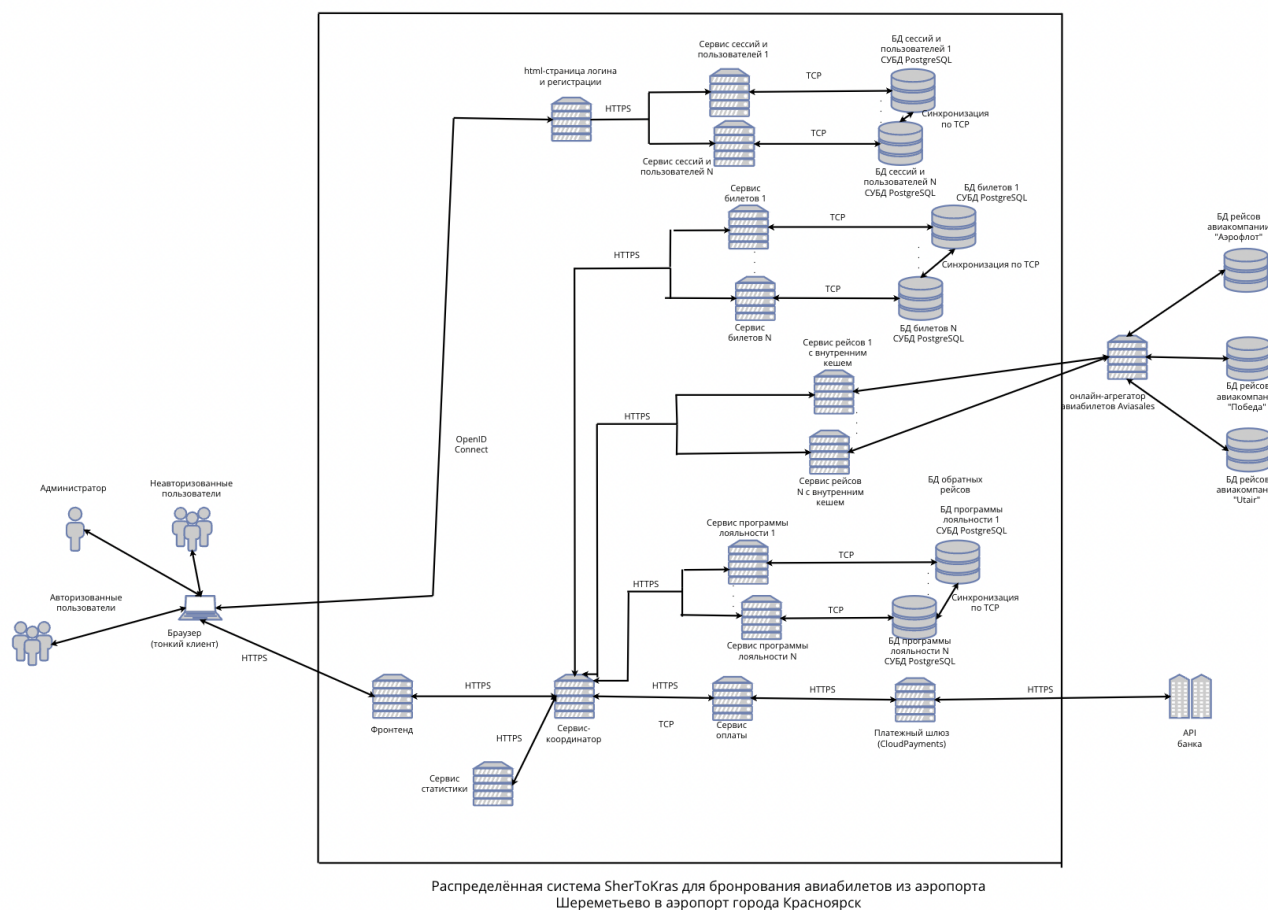


Рисунок 1 – Топология системы

Система должна состоять из 2 подсистем:

- подсистема авторизации;
- подсистема бронирования рейсов.

Подсистема бронирования рейсов должна состоять из фронтенда и шести сервисов, что наиболее целесообразно для реализации ее основного назначения.

Подсистема авторизации должна состоять из одного сервиса и html-страницы с полями для ввода логина и пароля, а также регистрации.

Взаимодействие подсистемы бронирования авиарейсов с подсистемой авторизации должно осуществляться по протоколу OpenID Connect.

Все сервисы подсистемы бронирования рейсов должны взаимодействовать друг с другом через сервис-координатор, запросы с фронтенда в том числе сначала должны приходить на сервис-координатор, а затем перенаправляться на нужный сервис.

Сервисы, используемые пользователями часто (сервис сессий и пользователей, сервис-координатор, сервис рейсов, сервис билетов, сервис программы лояльности) должны работать в нескольких экземплярах в разных географических позициях для обеспечения одинаковой задержки при загрузке страниц из разных географических позиций и, соответственно, меньшей нагрузки на сеть.

В рамках данной системы такие узлы размещения определяются на основе вероятных источников пользовательских запросов — это, прежде всего, Москва (аэропорт Шереметьево) и город Красноярск, в которых расположены аэропорты и в которые планируется выполнять авиаперелёты. Такой подход позволяет обеспечить более высокую производительность, устойчивость к сетевым сбоям и более комфортный пользовательский опыт.

В рамках одной географической позиции также могут быть использованы несколько работающих экземпляров сервиса. Нагрузка между сервисами может быть распределена с помощью алгоритма RoundRobin.

При этом некоторые из дублируемых сервисов обращаются к своей локальной копии базы данных, поэтому обязательной становится задача синхронизации данных между этими копиями. Синхронизация осуществляется по протоколу ТСР с использованием встроенных механизмов репликации, поддерживаемых большинством популярных СУБД. Такие механизмы позволяют обеспечить согласованность данных между географически распределёнными инстансами баз, сохраняя актуальность информации во всех точках доступа.

Фронтенд должен принимать запросы от пользователя по протоколу HTTP и возвращать ответ в виде HTML страниц, файлов стилей и java script.

Перечислим сервисы и их ответственность в системе.

1. Сервис-координатор отвечает за координацию запросов внутри системы. Для реализации балансировки запросов используется инфраструктура Kubernetes.
2. Сервис сессий и пользователей отвечает за сессию пользователей портала и реализует следующие функции:
 - регистрация пользователя (покупателя);
 - авторизация пользователя (вход, или «логин»);

- выход из сессии («логаут»).
- получение информации, изменение, удаление покупателя.

Сервис использует в своей работе базу данных. О каждом покупателе хранится следующая информация:

- логин (уникальное в рамках системы текстовое поле, максимум 256 символов);
- пароль (текстовое поле, не более 128 символов);
- имя и фамилия (каждое является текстовым полем, максимум 256 символов);
- Роль (пользователь / администратор);
- дата рождения.

3. Сервис программы лояльности реализует следующие функции:

- узнать баланс на счёте лояльности пользователя и историю изменения счёта лояльности с указанием для каждого события истории: суммы изменения, даты изменения, типа изменения и идентификатора билета, связанного с изменением;
- произвести начисление бонусных баллов при покупке билета;
- произвести списание бонусных баллов при возврате билета (при этом остаток бонусного счёта не может быть меньше 0).

Сервис программы лояльности использует в своей работе базу данных для хранения текущего баланса и истории изменений бонусного счёта каждого покупателя.

4. Сервис билетов реализует следующие функции:

- получить информацию о всех купленных билетах пользователя;
- получить информацию о билете пользователя с заданным идентификатором;
- отметить билет купленным или возвращённым.

Сервис билетов использует в своей работе базу данных. О каждом билете хранится следующая информация:

- уникальный в рамках системы идентификатор билета;
- логин пользователя, купившего билет;
- идентификатор рейса;
- цена билета;
- статус билета (оплачен / возвращён).

5. Сервис рейсов реализует следующие функции:

- получить информацию о всех рейсах;
- получить информацию о рейсе с заданным идентификатором.

Сервис рейсов не использует собственную базу данных для хранения информации о рейсах. При каждом пользовательском запросе сервис отправляет запрос к API онлайн-агрегатора Aviasales, который предоставляет актуальные данные о рейсах, тарифах и остатках мест. Для повышения производительности и снижения нагрузки на внешний API результаты таких запросов кешируются с использованием Redis.

При поступлении запроса формируется уникальный ключ кэша, включающий параметры маршрута (например, Шереметьево → Красноярск), дату вылета, количество пассажиров, класс обслуживания и другие необходимые детали. Если по этому ключу в Redis уже есть актуальный кешированный ответ, сервис немедленно возвращает данные из кэша. Если кеш отсутствует или истёк (TTL истёк), происходит обращение к API Aviasales, полученный результат сохраняется в Redis с заранее заданным временем жизни (TTL), и затем отправляется пользователю.

Выбор времени жизни кэша зависит от специфики авиаданных. В нашем сценарии устанавливается TTL кэша равный 15 минутам. Это означает, что результаты запроса к Aviasales будут сохраняться в Redis в течение 15 минут, что позволяет существенно снизить нагрузку на внешний API при повторных запросах от пользователей. При этом, поскольку динамические данные (например, цены, наличие мест или актуальный

статус рейса) могут изменяться даже в пределах этого интервала, в процессе оформления билета проводится финальная проверка: перед бронированием сервис всегда обращается к API Aviasales для уточнения и обновления динамических параметров. Таким образом, даже если пользователь изначально получил результаты из кеша, актуальность данных гарантируется за счёт дополнительного запроса при покупке, что обеспечивает баланс между быстродействием (благодаря Redis-кешированию) и точностью предоставляемой информации.

О каждом рейсе хранится следующая информация:

- уникальный в рамках системы идентификатор рейса;
- дата полёта;
- аэропорт отправления;
- аэропорт назначения;
- цена рейса.

6. Сервис оплаты отвечает за проведение операций по оплате и возврату билетов через внешний платёжный шлюз CloudPayments. Он получает запрос от координатора, регистрирует платёж и перенаправляет его в шлюз. После обработки транзакции CloudPayments сообщает результат, который фиксируется в системе и используется для завершения бронирования или возврата. Сервис обеспечивает надёжную и безопасную оплату с последующим уведомлением всех необходимых компонентов системы.
7. Сервис статистики отвечает за логирование событий во всей системе для осуществления возможности быстрого детектирования, локализации и воспроизведения ошибки в случае её возникновения.

Требования по реализации со стороны заказчика

1. Требуется использовать сервис-ориентированную архитектуру для реализации системы.
2. Требуется использование очереди сообщений Kafka для повышения отказоустойчивости системы.

3. Система состоит из микросервисов. Каждый микросервис отвечает за свою область логики работы приложения.
4. Взаимодействие между сервисами осуществляется посредством HTTP запросов.
5. Данные сервисов должны храниться в базе данных. Каждый сервис взаимодействует только со своей схемой данных. Взаимодействие сервисов происходит по технологии REST.
6. При недоступности систем портала должна осуществляться деградация функциональности или выдача пользователю сообщения об ошибке.
7. Необходимо предусмотреть авторизацию пользователей через интерфейс приложения.
8. Для авторизации использовать протокол OpenID Connect.
9. Для запросов, выполняющих обновление данных на нескольких узлах распределенной системы, в случае недоступности одной из систем, необходимо выполнять полный откат транзакции.
10. Приложение должно поддерживать возможность горизонтального и вертикального масштабирования за счет увеличения количества функционирующих узлов и совершенствования технологий реализации компонентов и всей архитектуры системы.

Функциональные требования по подсистемам

1. Фронтенд – это серверное приложение при разработке которого необходимо учитывать следующие факторы:
 - фронтенд должен принимать запросы по протоколу HTTPS и формировать ответы пользователям портала в формате HTML;
 - в зависимости от типа запроса фронтенд должен отправлять последовательные запросы в соответствующие бекенды;
 - запросы к бекендам необходимо осуществлять по протоколу HTTPS; данные необходимо передавать в формате JSON;

- требуется использование веб-сервиса Nginx для более быстрого возврата пользователям статического содержимого (изображения, таблиц стилей, JavaScript файлов), а также для балансировки запросов к нескольким экземплярам сервиса-координатора.
2. Сервис пользователей, сервис сессий, сервис программы лояльности, сервис рейсов и сервис билетов – это серверные приложения, которые должны отвечать следующим требованиям по разработке:
- обрабатывать запросы в соответствии со своим назначением, описанным в топологии системы;
 - принимать и возвращать данные в формате JSON по протоколу HTTPS;
 - осуществлять доступ к СУБД по протоколу TCP.
3. Сервис-координатор – это серверное приложение, которое должно отвечать следующим требованиям по разработке:
- обрабатывать запросы в соответствии со своим назначением, описанным в топологии системы;
 - принимать и возвращать данные в формате JSON по протоколу HTTPS;
 - использовать очередь для отложенной обработки запросов (например, при временном отказе одного из сервисов);
 - осуществлять деградацию функциональности в случае отказа некритического сервиса (зависит от семантики запроса);
 - существовать в нескольких экземплярах, чтобы координация запросов не была узким местом приложения;
 - уведомлять сервис статистики о событиях в системе.
4. Сервис статистики и сервис оплаты – это серверные приложения, которые должны отвечать следующим требованиям по разработке:
- обрабатывать запросы в соответствии со своим назначением, описанным в топологии системы;

- принимать и возвращать данные в формате JSON по протоколу HTTPS.

Сценарий взаимодействия фронтенда и бекендов

Рассмотрим, как взаимодействуют фронтенд и бекенды на примере выполнения запроса от пользователя на получение списка рейсов.

1. На фронтенд приходит данный запрос пользователя.
2. Фронтенд анализирует запрос пользователя и, если пользователь был авторизован ранее, получает из него JWT-токен. Далее выполняется запрос к сервису-координатору. Координатор проверяет корректность полученных данных (того, что данные поступили в ожидаемом формате, то есть в формате, соответствующем протоколу взаимодействия). Координатор отправляет запрос на сервис рейсов. В сервисе рейсов осуществляется проверка корректности полученных данных (того, что данные поступили в ожидаемом формате, то есть в формате, соответствующем протоколу взаимодействия) и проверка JWT-токен-а (проверка того, что токен был подписан известным серверу ключом и того, что срок действия токена ещё не истёк). Если он истёк или оказался некорректным, пользователю возвращается ошибка. При успешной проверке токена сервис возвращает список рейсов сервису-координатору, который в свою очередь возвращает результат на фронтенд.
3. Если ошибки нигде не произошло, то производится генерация HTML содержимого страницы ответа пользователю с использованием данных, полученных от сервиса рейсов. В ином случае генерируется страница с описанием ошибки.

Сценарий взаимодействия бекендов

Сервис-координатор — это серверное приложение, отвечающее следующим требованиям по разработке. Оно использует уже существующие решения, такие как Apache Kafka, которая необходима для сбора и обработки данных в реальном времени, что позволяет анализировать пользовательскую активность. Kafka способна обрабатывать высокие объёмы данных и легко масштабируется,

обеспечивая надёжную работу даже при значительных нагрузках. При этом сама Kafka может быть развернута в Kubernetes с использованием Helm-чартов или Strimzi Kafka Operator, что позволяет автоматически управлять жизненным циклом брокеров, настройками сети, хранением данных через StatefulSet и PersistentVolumeClaims, а также обновлениями и масштабированием.

Для управления Kafka из программного кода мы используем библиотеку на языке Python `confluent_kafka`.

Чтобы сервис-координатор получал внешние запросы, обычно настраивается Kubernetes Ingress, который позволяет описывать правила маршрутизации (на основе URL или доменного имени) и направлять HTTP-запросы от пользователей непосредственно к сервису-координатору. Таким образом, Kubernetes Ingress в сочетании с развернутой в Kubernetes Kafka обеспечивает полную инфраструктуру для высоконагруженной и надёжной обработки запросов: Ingress отвечает за внешние подключения, а Kafka – за асинхронную очередь сообщений, которая организует последовательную обработку поступающих задач даже при пиковых нагрузках.

Теперь рассмотрим сценарии взаимодействия бекендов при различных запросах пользователя:

1. Поиск рейсов пользователем

- **Поступление запросов (Kubernetes Ingress → Сервис-координатор)**
 - (a) Пользователь на фронтенде выбирает фильтры по которым будет делать запрос на просмотр рейсов и нажимает кнопку "Найти рейсы"
 - (b) Запрос отправляется в кластер Kubernetes и проходит через Kubernetes Ingress, где согласно настройкам маршрутизации (host/URL-правила) перенаправляется непосредственно на Сервис-координатор.
- **Публикация сообщения в очередь Kafka**

Сервис-координатор публикует сообщение в соответствующий топик Kafka (`flights_search_requests`).
- **Проверка доступности и получение данных (Сервис рейсов с Redis и прямым API)**

Сервис рейсов подписан на этот топик и, получив сообщение, проверяет свой кэш (TTL 15 минут):

- Если подходящие данные есть в Redis кэше, сервис рейсов сразу возвращает их.
- Если в кеше ничего нет, то мы обращаемся через API Aviasales напрямую за информацией по этому рейсу.
- Перед отправкой запроса в API Aviasales, данные, полученные от пользователя, необходимо предварительно преобразовать в новый формат, соответствующий требованиям и спецификациям Aviasales.

— **Обработка результата запроса**

- По завершении (независимо от успеха или ошибки) сервис рейсов формирует событие (FlightSearchResult), в котором указывает итоговый статус и данные (либо ошибку).
- Публикация события в отдельный топик «flights_search_responses».
- Сервис статистики подписан на этот топик, поэтому информации о результате запроса записывается в журнал.

— **Возврат данных на фронтенд**

- Фронтенд получает JSON-ответ, в котором содержится подробная информация по каждому найденному рейсу.
- На основе этой структуры JSON клиентская часть формирует динамический контент: пользователь видит список доступных перелётов, их стоимость, время вылета и прочие детали, отфильтрованные согласно исходным параметрам.

2. Бронирование рейса пользователем

— **Поступление запросов (Kubernetes Ingress → Сервис-координатор)**

- Пользователь на фронтенде выбирает рейс (маршрут, дату, класс обслуживания, количество пассажиров) и нажимает кнопку «Забронировать».

- Запрос отправляется в кластер Kubernetes и проходит через Kubernetes Ingress, где согласно настройкам маршрутизации (host/URL-правила) перенаправляется непосредственно на Сервис-координатор.
- **Публикация сообщения в очередь Kafka** Сервис-координатор публикует сообщение в соответствующий топик Kafka (flight_detail_requests).
- **Получение данных о рейсе**
 - Сервис рейсов подписан на этот топик и получив сообщение игнорируя кеш обращается к API Aviasales за детальной информацией о рейсе.
 - Перед отправкой запроса в API Aviasales, данные, полученные от пользователя, необходимо предварительно преобразовать в новый формат, соответствующий требованиям и спецификациям Aviasales.
- **Публикация события бронирования (Kafka)**
 - Если проверка подтверждает, что билет всё ещё доступен и данные актуальны, сервис-координатор формирует событие BookingRequestCreated с подробностями бронирования (маршрут, цена, данные пассажиров, ID запроса) и публикует его в топик Kafka (например, bookingRequests).
 - Это событие автоматически доступно для всех сервисов, подписанных на этот топик.
- **Обработка оплаты (Payment Service)**
 - Сервис оплаты слушает событие BookingRequestCreated (в топике booking_requests).
 - Иницирует транзакцию через платёжный шлюз (CloudPayments, банк и т.п.).
 - Публикует результат:
 - (a) При успехе: событие PaymentApproved в топик payment_approved.

- (b) При неудаче: событие `PaymentFailed` в топик `payment_failed`.
- Если на шаге финального обновления в Aviasales возникнет ошибка (описано в пункте об «Уведомлении Aviasales и финальном бронировании»), потребуется инициировать `refund` (возврат средств). В таком случае сервис оплаты будет слушать событие наподобие `AviasalesUpdateFailed` и проводить возврат (если политика возвратов это допускает), публикуя `PaymentRefundSucceeded` или `PaymentRefundFailed`.
- **Уведомление Aviasales и финальное бронирование**
 - После получения события `PaymentApproved` Сервис рейсов (или Сервис-координатор) запрашивает API Aviasales, уведомляя, что билет куплен и данные должны обновиться.
 - При успешном обновлении Aviasales сервис публикует событие `AviasalesUpdateSuccess` в топик `aviasales_update_success`:
 - (a) Сервис билетов подписан на `aviasales_update_success`, получает подтверждение и формирует финальную бронь (PNR, e-ticket).
 - (b) Публикует событие `ReservationConfirmed` (в топик `reservation_confirmed`), сигнализируя, что бронирование завершено.
 - (c) Пользователь получает уведомление от Сервиса уведомлений (слушающего `reservation_confirmed`).
 - Если Aviasales возвращает ошибку (не удалось обновить информацию):
 - (a) Сервис рейсов/координатор публикует событие `AviasalesUpdateFailed` в топик `aviasales_update_failed`.
 - (b) Сервис оплаты или координатор, подписанный на `aviasales_update_failed`, инициирует возврат средств (`refund`) путём публикации команды `PaymentRefundRequested`.
 - (c) В случае успеха возврата публикуется `PaymentRefundSucceeded`, при сбое — `PaymentRefundFailed`.

- (d) Сервис уведомлений информирует пользователя о том, что бронирование не состоялось и платёж возвращён (или не возвращён при `PaymentRefundFailed`), а Сервис статистики фиксирует негативный сценарий.

— **Пост-обработка**

- Сервис лояльности также получает событие `ReservationConfirmed` (или отдельное событие `PointsAwarded`, инициированное после бронирования) для начисления бонусных миль/баллов пользователю.
- Сервис статистики подписан на ВСЕ ключевые топики (например, `booking_requests`, `payment_approved`, `payment_failed`, `reservation_confirmed`, `aviasales_updates`):
 - (a) Он собирает информацию о количестве успешных и неудачных бронирований, времени отклика на каждое событие, динамике цен и других метриках.
 - (b) Такая сборка статистики обеспечивает аналитическую базу для оптимизации процессов и оценки качества сервиса.

3. Просмотр пользователем личного кабинета

- **Поступление запросов (Kubernetes Ingress → Сервис-координатор)**
 - Пользователь заходит на страницу личного кабинета.
 - Запрос отправляется в кластер Kubernetes и проходит через Kubernetes Ingress, где согласно настройкам маршрутизации (`host/URL`-правила) перенаправляется непосредственно на Сервис-координатор.
- **Публикация сообщения в очередь Kafka** Сервис-координатор формирует событие, содержащее детали запроса, и публикует его в топик Kafka, например, `personal_cabinet_requests`.
- **Взаимодействие с сервисом авторизации**
 - Сервис авторизации, подписанный на соответствующий топик или получающий запрос от Сервис-координатора, проверяет наличие данных пользователя.

- Если данные пользователя найдены:
 - * Сервис авторизации публикует событие (например, `UserDataFound`) с информацией о пользователе в топик, на который подписан Сервис-координатор.
 - * Сервис-координатор получает событие, отправляет данные о пользователе на фронтенд, обеспечивая показ личного кабинета.
- Если данных пользователя не найдено:
 - * Сервис авторизации инициирует перенаправление (или публикует соответствующее событие, например, `UserDataMissing`), что приводит к переходу пользователя на страницу регистрации.
- **Публикация данных в сервис статистики**
 - Независимо от результата (успех или отсутствие данных), событие о результате запроса публикуется в топик для статистики, например, `personal_cabinet_responses`, который подписан на этот.
- **Возврат данных на фронтенд**
 - Сервис-координатор, подписанный на `personal_cabinet_responses`, получает событие об итоговом статусе:
 - * Если это `UserDataFound`, координатор передаёт на фронтенд JSON-объект с данными пользователя (имя, фамилия, e-mail и т.п.).
 - * Если это `UserDataMissing`, координатор перенаправляет пользователя (или возвращает ответ фронтенду), указывая, что необходимо перейти на страницу регистрации.

4. Просмотр пользователем забронированных им рейсов

- **Поступление запросов (Kubernetes Ingress → Сервис-координатор)**
 - Пользователь находясь на странице личного кабинета, нажимает на пункт меню "Мои бронирования".

- Запрос включает JWT-токен, который передаётся через Ingress в Сервис-координатор.
- **Получение user_id через Сервис авторизации**
 - Сервис-координатор извлекает JWT-токен и отправляет его в Сервис авторизации.
 - Сервис авторизации проверяет токен и возвращает уникальный идентификатор пользователя (user_id).
- **Публикация запроса в очередь Kafka**
 - Сервис-координатор формирует сообщение запроса, включающее user_id и другие необходимые параметры.
 - Сообщение публикуется в топик Kafka, например, user_bookings_requests.
- **Обработка запроса (Сервис билетов)**
 - Сервис билетов, подписанный на user_bookings_requests, получает сообщение.
 - Используя user_id, сервис билетов осуществляет поиск бронирований в своей базе данных.
 - Результат (список бронирований) формируется и публикуется в топик user_bookings_responses.
- **Отправка данных в Сервис статистики**

Сервис статистики подписан на топик user_bookings_responses, и при публикации сообщения в этот топик, информация передается в сервис статистики для журналирования.
- **Возврат данных на фронтенд**
 - Фронтенд получает JSON-ответ с массивом забронированных рейсов:
 - * Если массив пустой, пользователю отображается сообщение «Нет активных бронирований».
 - * Если в JSON содержатся данные, пользователь видит список своих текущих рейсов с ключевой информацией (номер рейса, время вылета и т.д.).

Пользовательский интерфейс

Для реализации пользовательского интерфейса должен быть использован подход MVC (Model-View-Controller). Этот подход к проектированию интерфейса является популярным шаблоном проектирования, который помогает разделить логику приложения на три основных компонента: Модель (Model), Представление (View) и Контроллер (Controller). Этот подход позволяет улучшить структуру приложения, облегчить его тестирование и управление, а также разработка фронтенда и бекенда могут быть полностью разделены между собой, то есть можно вести независимую разработку.

Пользовательский интерфейс в разрабатываемой системе должен обладать следующими характеристиками:

- Адаптивность к размеру экрана устройства пользователя – пользовательский интерфейс «подстраивается» под всевозможные размеры экранов устройств: мобильных телефонов, планшетов, ноутбуков и т.д.
- Кроссбраузерность – способность интерфейса работать практически в любом браузере любой версии.
- «Плоский» дизайн» – дизайн, в основе которого лежит идея отказа от объемных элементов (теней элементов, объемных кнопок и т.д.) и замены их плоскими аналогами. Данная технология становится популярной в последнее время по ряду причин:
 1. Простота поддержки.
 2. Упор на развитую функциональность, а не на красоту интерфейса.
 3. Сокращение затрат на разработку дизайна.
- Расширяемость – возможность легко расширять и модифицировать пользовательский интерфейс. Интерфейс спроектирован таким образом, чтобы его можно было дополнять новыми компонентами, не нарушая текущую структуру и логику взаимодействия. Модульность вёрстки и кода: каждая функциональная часть (виджет, панель, меню) оформлена как самостоятельный блок (компонент), который можно добавлять, удалять или заменять без полной перестройки всего интерфейса.

- Интуитивно понятный интерфейс – все кнопки имеют подписи при наведении на них, многие содержат иконки, облегчающие восприятие пользователем.

Пользовательский интерфейс в разрабатываемой системе представляет собой web-интерфейс, доступ к которому осуществляется через браузер (тонкий клиент). Страница системы состоит из «шапки», основной части.

Обобщенно структуру страниц системы можно представить следующим образом:

- главная страница с поиском авиабилетов;
- страница подробной информацией о билете и кнопкой покупки;
- личный кабинет пользователя (информация о пользователе с возможностью ее изменить);
- страница с купленными билетами (с возможностью вернуть еще не использованные билеты);
- страница программы лояльности (история операций бонусного счета, остаток на бонусном счёте);
- страница входа;
- страница регистрации;
- страница со статистикой запросов в приложении (доступно администраторам);

Требования к дизайну

Дизайн веб-сайта системы бронирования авиабилетов должен быть минималистичным, удобным и интуитивно понятным. В оформлении системы бронирования предполагается использовать сине-белую цветовую гамму — такой выбор символизирует небо и облака, тесно связанные с авиаперелётами. Сочетание синего фона и белых элементов подчёркивает тематику сайта и формирует лёгкое, «воздушное» восприятие интерфейса, что соответствует идее комфортных авиапутешествий. Интерфейс должен быть адаптивным для удобной работы как на компьютерах, так и на мобильных устройствах.

1. Главная страница бронирования авиабилетов

Цель страницы: представить пользователям сервис бронирования авиабилетов, предоставить быстрый доступ к основным функциям (поиск, авторизация)

- **Шапка (header):**

- Крупный текст «Путешествовать стало проще», который привлекает внимание и подчёркивает идею быстрого/лёгкого бронирования.

- **Основной блок (середина):**

- Выбор города вылета (по умолчанию «Москва (SVO)»).
- Поле «Куда летим?» — текстовое поле (placeholder), куда пользователь вводит или выбирает аэропорт назначения.

- **Подвал (footer):**

- Набор иконок внизу экрана (примерно три-четыре пункта):
- Значок «Поиск» (активен на текущей странице).
- Другие иконки (например, «Билеты», «Профиль», «Дополнительно»), обеспечивающие доступ к другим страницам.

2. Страница авторизации

Цель страницы: Предоставить пользователю удобный способ войти в свой профиль для бронирования авиабилетов. Упростить процесс входа за счёт интеграции с социальными сетями (Google, Facebook и т.д.).

- **Шапка (header):**

- **Фоновое изображение:** Верхнюю часть экрана занимает яркий сине-белый фон с облаками, на котором изображён самолёт в полёте. Сцена визуально передаёт тему авиапутешествий и задаёт общий «воздушный» стиль приложения.
- **Заголовок "Вход"/"Регистрация":** Размещён по центру, поверх фона с самолётом. Этот заголовок чётко указывает на назначение страницы — авторизацию пользователя.

- **Основной блок (середина):**

- **Форма авторизации:**
 - (а) **Поле «Введите e-mail»:** текстовое поле для ввода электронной почты.
 - (b) **Поле «Введите пароль»:** скрытое поле для пароля.
 - (с) **Кнопка «Login»:** расположена сразу под полями ввода, обычно выделена синим или контрастным цветом для привлечения внимания.
- **Быстрый вход через соцсети:** Прямо под кнопкой «Login» отображаются иконки (Google, Facebook), которые позволяют авторизоваться, используя учётные записи из популярных социальных сетей.
- **Подвал (footer):**
 - **Ссылка для регистрации:** надпись «Нет аккаунта? Зарегистрироваться», обычно располагается внизу экрана. Пользователь может нажать на «Зарегистрироваться» и перейти к форме создания нового профиля.

3. Страница личного кабинета

Цель страницы: Предоставить пользователю централизованное пространство для управления своей учётной записью в сервисе бронирования авиабилетов, где он может просматривать и редактировать личные данные, получать доступ к бронированиям, бонусной программе и дополнительным настройкам, а также обращаться в службу поддержки — обеспечивая тем самым персонализированный и удобный опыт использования сервиса.

- **Шапка (header)**
 - (а) **Отображение учётной записи:**
 - В верхней части экрана указаны e-mail пользователя и дополнительная информация (город «Москва, Россия»).
 - Значок «>» справа может указывать на переход к детальным настройкам профиля или выпадающему меню.
 - (b) **Фоновый стиль:**

- Шапка выделена цветом или визуальной границей, в целом выдерживая общую сине-белую цветовую гамму, характерную для авиасервиса.
- Здесь же может быть кнопка «назад» или «заккрыть», если это боковая панель (меню).
- **Основной блок (середина)**
 - **Профиль:** Переход к деталям учётной записи, где можно редактировать личные данные, фотографию и т.д.
 - **Бонусная программа:** Отдельный раздел для накопления миль/баллов и просмотра текущего баланса.
 - **Мои бронирования:** История и статус приобретённых билетов, доступ к посадочным талонам, деталям перелётов.
 - **Данные пассажира:** Ввод или редактирование паспортных данных, предпочтений (бортовое питание, место), которые будут использоваться при следующих бронированиях.
 - **Поддержка:** Ссылка на FAQ, чат или форму отправки запросов в службу поддержки.
 - **Контакты:** Адреса, телефоны и e-mail, по которым можно связаться со службой сервиса.
 - **Настройки:** Дополнительные параметры приложения: язык интерфейса, уведомления, персональные предпочтения.
- **Подвал (footer)**
 - Логотип компании или краткое упоминание сервиса в самом низу.
 - Версию приложения или года действия лицензии.
 - Кнопку выхода (лог-аут), если она не размещена в основном списке.

4. Страница мои бронирования

Цель страницы: Позволить пользователю быстро просмотреть все ранее оформленные (или предстоящие) авиабилеты, узнать детали по каждому рейсу (аэропорт, дату/время вылета и прилёта, авиакомпанию,

класс обслуживания), а при необходимости перейти к дополнительной информации или управлению бронированием.

— **Шапка (header)**

- (a) **Заголовок «Мои бронирования»** Крупным шрифтом в верхней части экрана указано название страницы, чтобы пользователь понимал, что это список всех его приобретённых перелётов.
- (b) **Кнопка назад или меню** В левом верхнем углу может располагаться стандартная «стрелка» для возврата на предыдущий экран (или иконка меню, если дизайн приложения предполагает боковую навигацию).

— **Основной блок (середина)**

- (a) **Список (карточки) забронированных рейсов** : каждая карточка включает в себе следующую информацию:
 - Маршрут (указывающий аэропорт вылета/прилёта)
 - Дата и время вылета
 - Дата и время прилета
 - Информация об авиакомпании
- (b) **Множественные записи:** Если у пользователя несколько бронирований, они идут списком (карточками), каждое содержит уникальные данные: даты, время, авиакомпанию, маршрут.
- (c) **Кликабельность:** Каждая карточка может вести на детальную страницу бронирования (при нажатии), где пользователь увидит посадочные талоны, правила возврата, дополнительные опции (покупка багажа, выбор места и т.п.).

— **Подвал (footer)**

— **Навигационные элементы**

- (a) Приложение имеет стандартную нижнюю панель навигации (с иконками «Главная», «Поиск», «Мои бронирования», «Профиль» и т.д.).
- (b) Текущая вкладка («Мои бронирования») может быть под-

свечена, чтобы пользователь знал, что находится на странице бронирований.

Требования к составу и параметрам технических средств

Все серверные приложения должны потреблять суммарно не более 2 Гбайт оперативной памяти и работать на сервере с процессором Intel(R) Core(TM) i7- 10510U CPU 1.80GHz.

Требования к надежности

Система должна работать в соответствии с данным техническим заданием без перезапуска. Необходимо использовать «зеркалируемые серверы» для всех подсистем, которые будут держать нагрузку в случае сбоя до тех пор, пока основной сервер не восстановится.

Требования к документации

Исполнитель должен подготовить и передать Заказчику следующие документы:

- руководство по развертыванию Системы;
- руководство администратора Системы;
- руководство для покупателя по использованию Системы.

Проектирование системы

Концептуальный дизайн

Концептуальная модель системы в нотации IDEF0

Для создания функциональной модели портала, отражающей его основные функции и потоки информации наиболее наглядно использовать нотацию IDEF0. На рисунке 1 приведена концептуальная модель системы. На рисунке 2 представлена декомпозиция функциональной модели системы.



Рисунок 1 – Концептуальная модель в нотации IDEF0

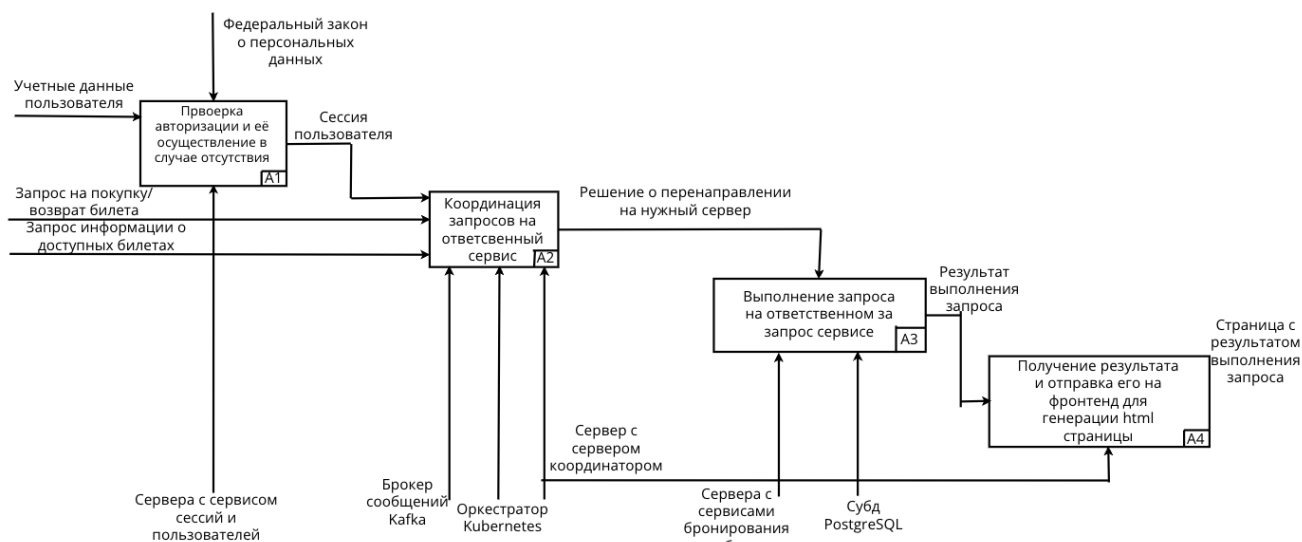


Рисунок 2 – Детализированная концептуальная модель в нотации IDEF0

Сценарии функционирования системы

Регистрация пользователя

1. Пользователь нажимает на кнопку «Зарегистрироваться» в интерфейсе.
2. Пользователь перенаправляется на страницу, которая содержит поля для заполнения его данных.
3. Пользователь вводит данные в форму и для завершения регистрации нажимает на кнопку «Зарегистрироваться», тем самым подтверждая верность своих данных, а также согласие на их обработку и хранение.
4. Если пользователь с введенным для регистрации логином, почтой или номером телефона уже существует, то клиент получает сообщение об ошибке. При успешной регистрации клиент попадает на страницу для просмотра авиабилетов.

Авторизация клиента

1. Пользователь нажимает на кнопку «Авторизоваться» в интерфейсе.
2. Пользователь перенаправляется на страницу авторизации, которая содержит поля для заполнения логина и пароля.

3. Пользователь завершает работу с формой авторизации нажатием кнопки «Войти».
4. При обнаружении ошибки в данных, пользователь получает сообщение об ошибке; при совпадении данных с записью в базе данных аккаунтов пользователь получает доступ к системе и перенаправляется на страницу для просмотра авиабилетов.

Поиск и просмотр билетов по заданным фильтрам пользователем

1. Функциональность доступна как авторизованным, так и неавторизованным пользователям.
2. Пользователь вводит параметры поиска в форму:
 - пункт отправления;
 - пункт назначения;
 - дата рейса;
 - количество пассажиров;
 - класс обслуживания (эконом, бизнес и т.д.).
3. После ввода параметров пользователь инициирует поиск.
4. Результаты поиска отображаются на той же странице в виде набора карточек рейсов.
5. Каждая карточка содержит полную информацию о рейсе: маршрут, дата и время отправления и прибытия, длительность, стоимость и доступность мест.
6. Для каждого найденного рейса предоставляется кнопка «Забронировать», при нажатии на которую пользователь переходит на страницу оформления бронирования.

Покупка билета

1. После того как пользователь выполнил поиск рейсов, он нажимает на кнопку «Забронировать», расположенную напротив выбранного рейса.

2. Пользователь перенаправляется на отдельную страницу с детальной информацией по выбранному рейсу, включая обновлённую цену и условия.
3. В нижней части страницы отображается кнопка «*Оплатить*».
4. После нажатия на кнопку «*Оплатить*», пользователь перенаправляется на страницу платёжного шлюза CloudPayments.
5. Пользователь выбирает способ оплаты, вводит данные и подтверждает транзакцию.
6. В случае успешной оплаты и положительного ответа от банка, пользователь попадает на страницу постобработки.
7. На этой странице отображается уведомление: **«Билет успешно забронирован»**.
8. Пользователю предоставляются две кнопки:
 - «*Мои бронирования*» — переводит пользователя на страницу активных бронирований.
 - «*На главную*» — возвращает пользователя на главную страницу для повторного поиска рейсов.

Возврат купленного билета

1. Для просмотра купленных билетов пользователь переходит на страницу «Билеты», нажимая соответствующую кнопку в выезжающем боковом меню.
2. Пользователь попадает на страницу, на которой видит все свои купленные и сданные билеты.
3. У каждого купленного билета есть кнопка «Сдать билет», на которую нажимает пользователь.
4. В появившемся модальном окне подтверждения с подробной информацией о выбранном билете пользователь нажимает на кнопку «Подтвердить».

5. Пользователь видит измененный статус билета на «Билет сдан» и обновленный баланс бонусного счета.

Просмотр пользователем личного кабинета

1. Пользователь нажимает на вкладку «Личный кабинет» в нижней панели интерфейса.
2. Система проверяет, авторизован ли пользователь:
 - Если пользователь неавторизован, происходит перенаправление на страницу с формой регистрации/входа.
 - Если пользователь авторизован, осуществляется переход на страницу личного кабинета.
3. На странице личного кабинета отображается персональная информация пользователя:
 - Электронная почта и город.
 - Список разделов: профиль, бонусная программа, мои бронирования, данные пассажира, поддержка, контакты, правила и соглашения, гарантии, настройки.
4. Пользователь может перейти в любой из представленных разделов, нажав на соответствующий пункт.

Просмотр активных бронирований пользователем

1. Пользователь, находясь в личном кабинете, нажимает на вкладку «*Мои бронирования*».
2. Система проверяет, авторизован ли пользователь:
 - Если пользователь неавторизован, доступ к разделу невозможен (по умолчанию пользователь не может попасть в личный кабинет).
 - Если пользователь авторизован, происходит переход на страницу активных бронирований.
3. На странице отображается список всех активных бронирований пользователя:

- Каждое бронирование представлено в виде отдельной карточки с информацией о рейсе (маршрут, дата, пассажиры, статус и т.д.).
 - Рядом с каждым бронированием присутствует кнопка «Отменить бронирование», позволяющая отменить бронь.
4. Если у пользователя отсутствуют активные бронирования, на экране отображается водяной знак с текстом «Нет активных бронирований».

Диаграммы прецедентов

Для детальной разработки портала используется унифицированный язык моделирования UML. В системе выделены 2 роли: администратор и пользователь (авторизованный и неавторизованный). На рисунках 4-3 представлены диаграммы прецедентов для выделенных ролей и описаны сценарии функционирования наиболее значимых прецедентов.

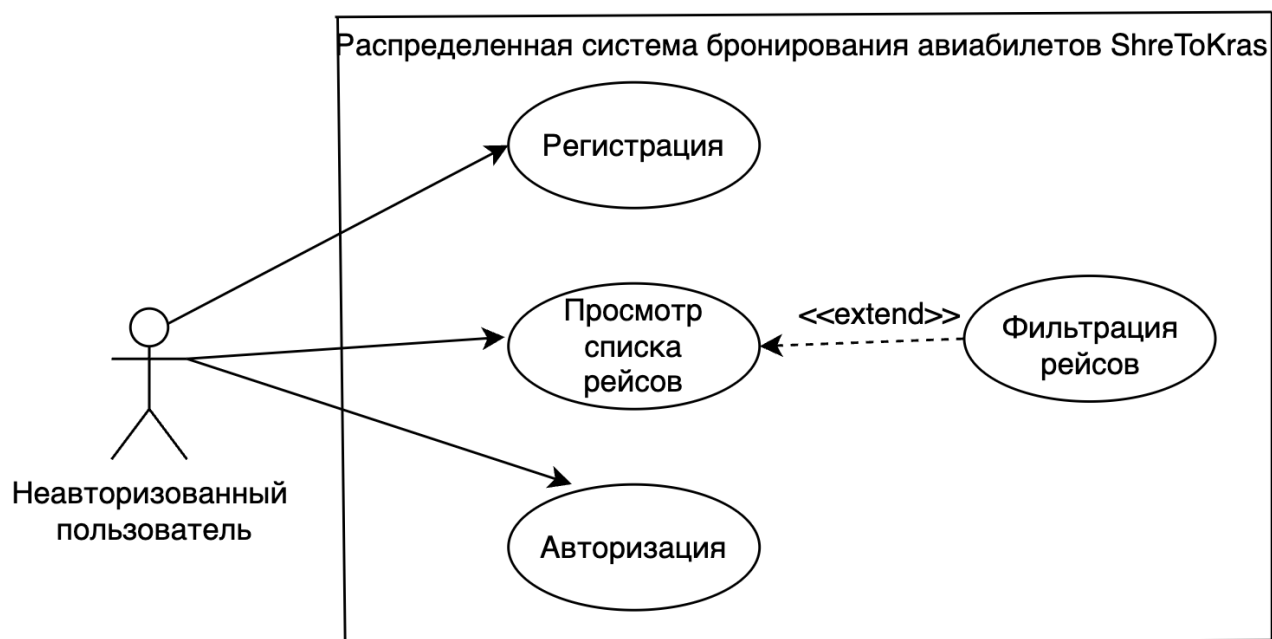


Рисунок 3 – Диаграмма прецедентов с точки зрения Неавторизованного пользователя

Спецификация сценария «регистрация пользователя»

Нормальный ход сценария

Действие актера	Отклик системы
Пользователь нажимает кнопку "Регистрация" на стартовой странице	Система предоставляет пользователю форму для регистрации, в которой нужно заполнить фамилию, имя, логин, пароль, номер телефона, электронную почту
Пользователь заполняет поля формы (фамилию, имя, логин, пароль, номер телефона, электронную почту), согласно необходимому формату, и нажимает кнопку "Зарегистрироваться"	Данные пользователя регистрируются в системе. Пользователь перенаправляется на главную страницу портала с формой для поиска рейсов.

Альтернативный ход сценария

Действие актера	Отклик системы
Пользователь нажимает кнопку "Регистрация" на стартовой странице	Система предоставляет пользователю форму для регистрации, в которой нужно заполнить фамилию, имя, логин, пароль, номер телефона, электронную почту
Пользователь заполняет поля формы (фамилию, имя, логин, пароль, номер телефона, электронную почту) и нажимает кнопку "Зарегистрироваться" но в одном из них введенные данные, не соответствуют требуемому формату	На странице регистрации под формой появляется сообщение о неверном формате заполненных данных

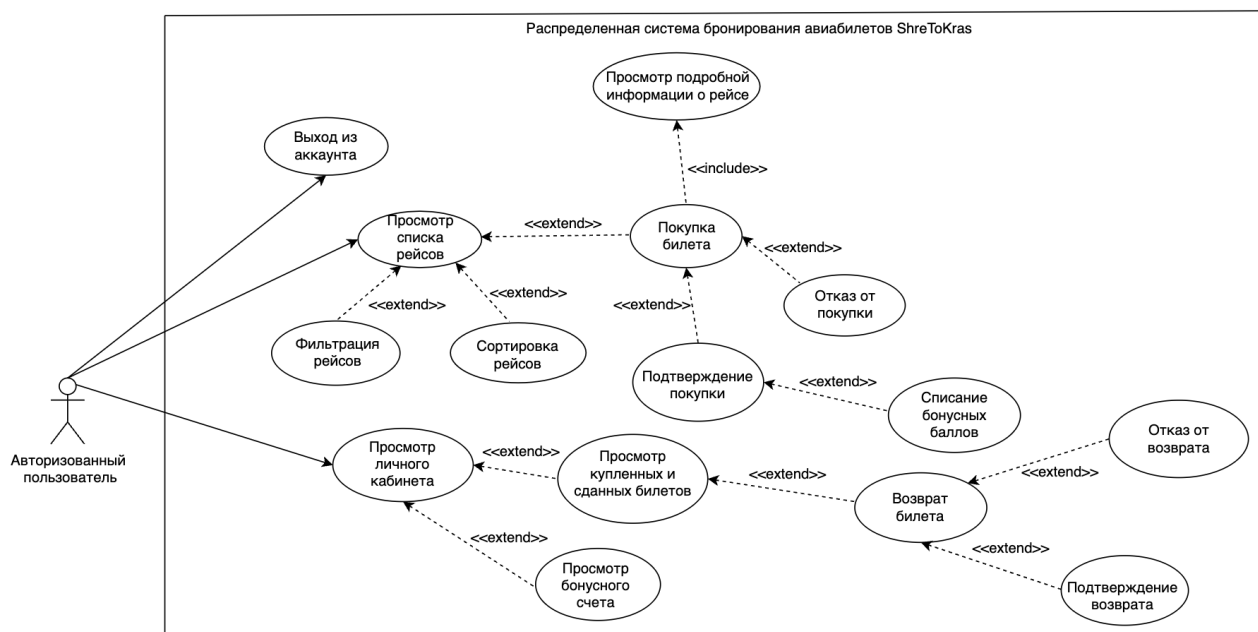


Рисунок 4 – Диаграмма прецедентов с точки зрения Авторизованного пользователя

Спецификация сценария «авторизация пользователя»

Нормальный ход сценария

Действие актера	Отклик системы
Пользователь нажимает кнопку "Авторизация" на стартовой странице	Система предоставляет пользователю форму для авторизации, в которой нужно заполнить логин, пароль
Пользователь заполняет поля формы (логин, пароль) и нажимает кнопку "Войти"	Система проверяет правильность введенных пользователем данных и, если данные правильные, перенаправляет пользователя на главную страницу портала с формой для поиска рейсов

Альтернативный ход сценария

Действие актера	Отклик системы
-----------------	----------------

Действие актера	Отклик системы
Пользователь нажимает кнопку "Авторизация" на стартовой странице	Система предоставляет пользователю форму для авторизации, в которой нужно заполнить логин, пароль, номер телефона, электронную почту
Пользователь заполняет поля формы и нажимает кнопку "Войти" но при этом преднамеренно или нет совершает ошибку при вводе логина и пароля	Система отказывает в доступе и на странице авторизации под формой появляется сообщение о том, что пользователь с данной комбинацией логина и пароля не найден

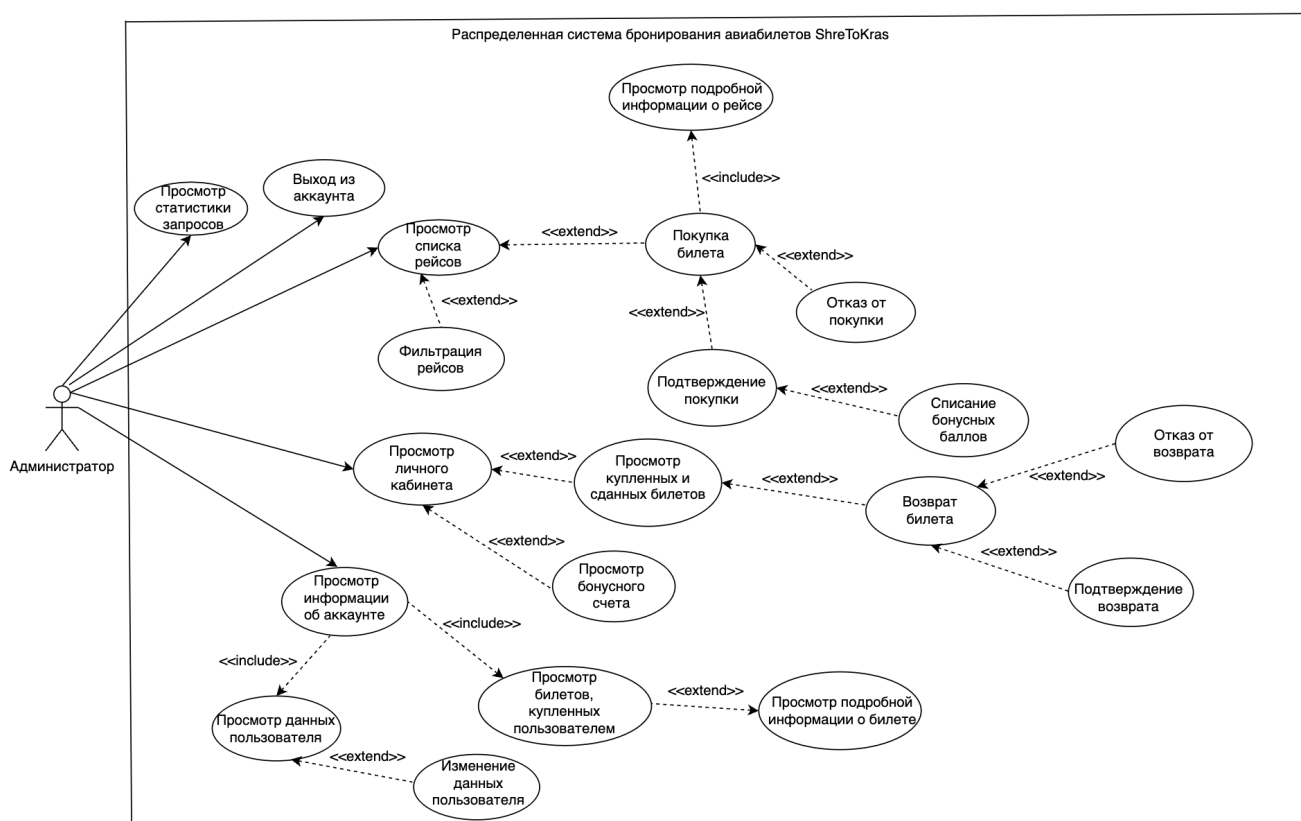


Рисунок 5 – Диаграмма прецедентов с точки зрения Администратора

Спецификация сценария «покупка билета»

Нормальный ход сценария.

Действие актера	Отклик системы
-----------------	----------------

Действие актера	Отклик системы
После успешной авторизации пользователь попадает на главную страницу портала, на которой представлена форма для поиска рейсов. Форма содержит в себе несколько полей для задания параметров поискового запроса рейсов (номер полета, аэропорт отправления/прибытия, время вылета/прилета, минимальная/максимальная цена). Пользователь должен заполнить как минимум одно из полей формы, чтобы система могла сформировать поисковый запрос рейсов.	Система формирует запрос на получение списка рейсов по заданным параметрам, передаёт его в сервис поиска рейсов, получает список соответствующих рейсов и возвращает данные, которые на клиентской стороне отображаются в виде карточек с информацией о рейсе и кнопкой для покупки.
Пользователь нажимает на кнопку "Забронировать" в карточке рейса для бронирования билета.	Система делает запрос на проверку наличия билетов по данному рейсу, после чего пользователь перенаправляется на страницу с подробной информацией о рейсе и кнопкой "Оплатить". Предварительно система проверяет, есть ли у пользователя бонусные баллы, и в зависимости от этого под информацией о рейсе появляется switch-кнопка "Списать бонусные баллы".
Пользователь имеет возможность нажать на кнопку "Использовать бонусные баллы"	Система обновляет финальную стоимость билета на странице с подробной информацией о билете в соответствие с информацией о используемых пользователем бонусных баллах.
Пользователь нажимает кнопку "Оплатить" для совершения оплаты билета.	Система перенаправляет пользователя на страницу с оплатой через платежный шлюз.

Действие актера	Отклик системы
Пользователь вводит данные своей карты и нажимает кнопку "Оплатить"	Платежный шлюз отправляет запрос в соответствующий банк для подтверждения транзакции.
Пользователь перенаправляется на страницу платежного шлюза для ожидания завершения транзакции.	После успешного завершения транзакции система перенаправляет пользователя на страницу с рейсами и отображает модальное окно с информацией о том, что рейс забронирован и сколько бонусных баллов было зачислено на бонусный счет пользователя.

Альтернативный ход сценария.

Действие актера	Отклик системы
После успешной авторизации пользователь попадает на главную страницу портала на которой представлена форма для поиска рейсов. Форма содержит в себе несколько полей для задания параметров поискового запроса рейсов (номер полета, аэропорт отправления/прибытия, время вылета/прилета, минимальная/максимальная цена). Пользователь должен заполнить как минимум одно из полей формы, чтобы система могла сформировать поисковый запрос рейсов.	Система формирует запрос на получение списка рейсов по заданным параметрам, передаёт его в сервис поиска рейсов, но по данному запросу не находится ни одного рейса. Система отправляет сообщение об этом на фронтент, и перед пользователем появляется сообщение "По вашему запросу рейсов не найдено"

Альтернативный ход сценария.

Действие актера	Отклик системы
После успешной авторизации пользователь попадает на главную страницу портала на которой представлена форма для поиска рейсов. Форма содержит в себе несколько полей для задания параметров поискового запроса рейсов (номер полета, аэропорт отправления/прибытия, время вылета/прилета, минимальная/максимальная цена). Пользователь должен заполнить как минимум одно из полей формы, чтобы система могла сформировать поисковый запрос рейсов.	Система формирует запрос на получение списка рейсов по заданным параметрам, передаёт его в сервис поиска рейсов, получает список соответствующих рейсов и возвращает данные, которые на клиентской стороне отображаются в виде карточек с информацией о рейсе и кнопкой для покупки.
Пользователь нажимает на кнопку "Забронировать" в карточке рейса для бронирования билета.	Система делает запрос на проверку наличия билетов по данному рейсу, после чего пользователь перенаправляется на страницу с подробной информацией о рейсе и кнопкой "Оплатить". Предварительно система проверяет, есть ли у пользователя бонусные баллы, и в зависимости от этого под информацией о рейсе появляется switch-кнопка "Списать бонусные баллы".
Пользователь имеет возможность нажать на кнопку "Использовать бонусные баллы" если у него ненулевой бонусный баланс.	Система обновляет финальную стоимость билета на странице с подробной информацией о билете в соответствии с информацией о используемых пользователем бонусных баллах.

Действие актера	Отклик системы
Пользователь нажимает кнопку "Оплатить" на странице с подробной информацией о билете, но не авторизован. Это может произойти, например, если срок действия пользовательской сессии истёк, куки были удалены или произошёл сбой в хранении токена авторизации.	Система перенаправляет пользователя на страницу авторизации.
Пользователь вводит данные для авторизации (логин и пароль) и нажимает кнопку "Войти".	Система проверяет правильность введенных данных пользователя. Если данные правильные, перенаправляет пользователя на страницу с оплатой через платежный шлюз.
Пользователь вводит данные своей карты и нажимает кнопку "Оплатить".	Платежный шлюз отправляет запрос в соответствующий банк для подтверждения транзакции.
Платежный шлюз получает подтверждение от банка и завершает операцию оплаты.	Система перенаправляет пользователя на страницу с рейсами и отображает модальное окно с информацией о том, что рейс забронирован и сколько бонусных баллов было зачислено на бонусный счет пользователя.

Логический дизайн

Диаграммы классов

Иерархии классов для разработки серверных приложений представлены в виде диаграммы классов:

- сервиса рейсов – на рисунке 6;
- сервиса билетов – на рисунке 7;
- сервиса сессий и пользователей – на рисунке 8;
- сервиса программы лояльности – на рисунке 9;

- сервиса статистики — на рисунке 12.

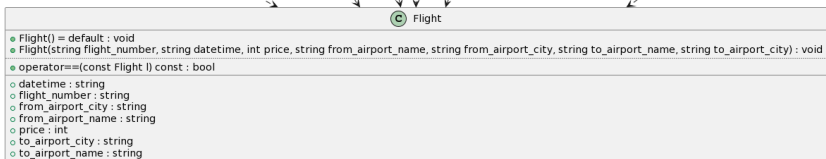


Рисунок 6 – Диаграмма классов сервиса рейсов

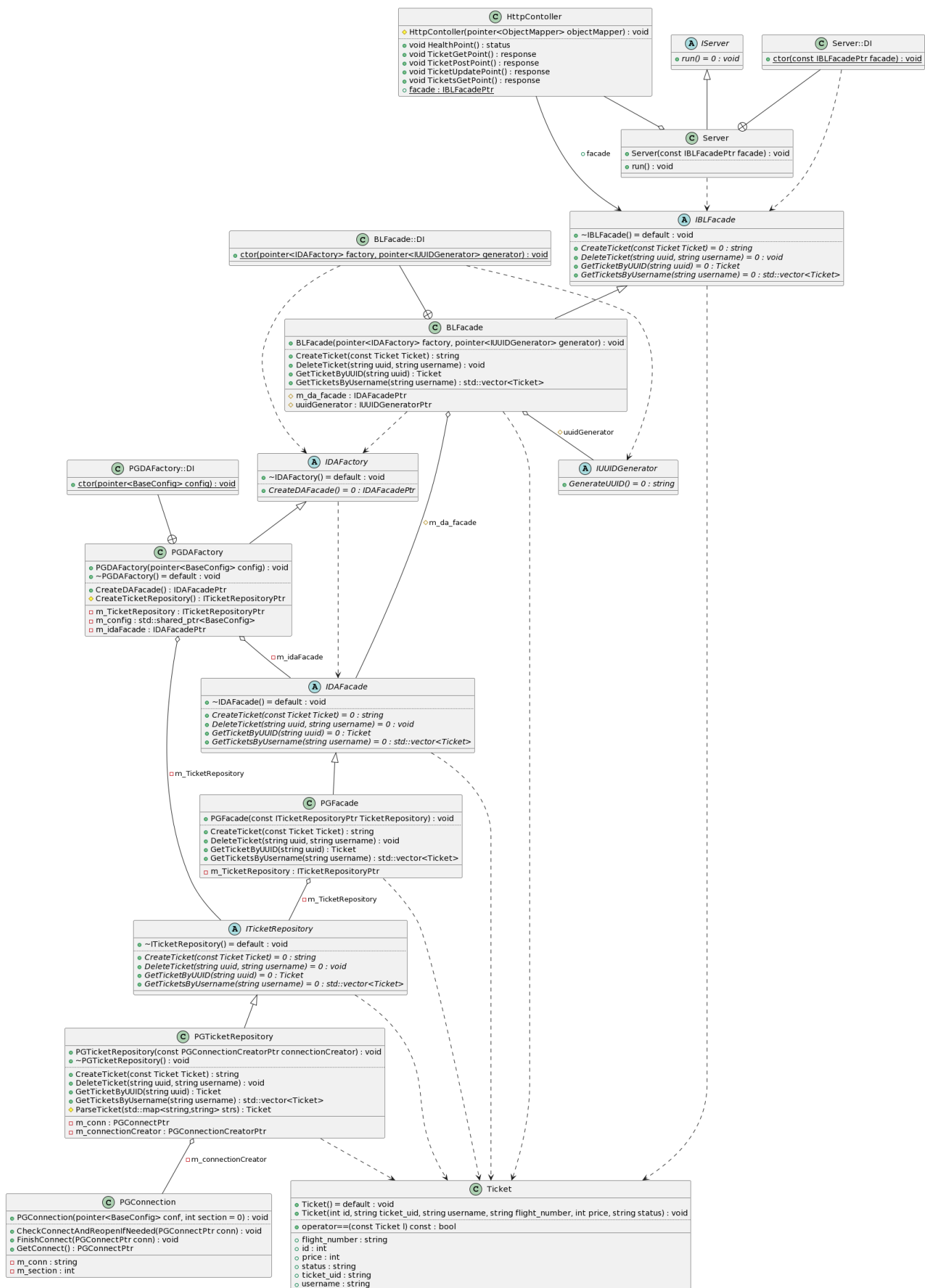


Рисунок 7 – Диаграмма классов сервиса билетов

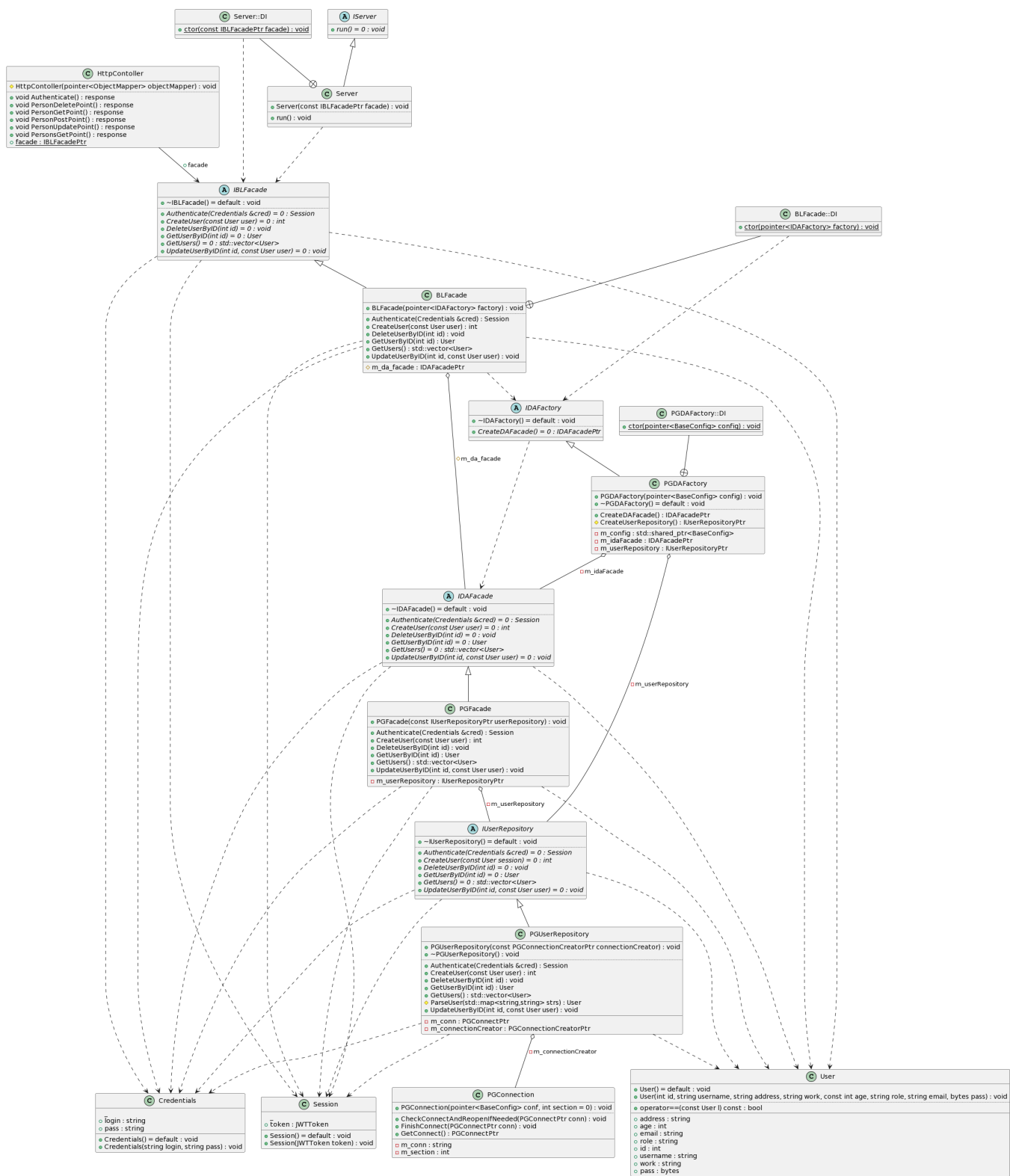


Рисунок 8 – Диаграмма классов сервиса сессий и пользователей

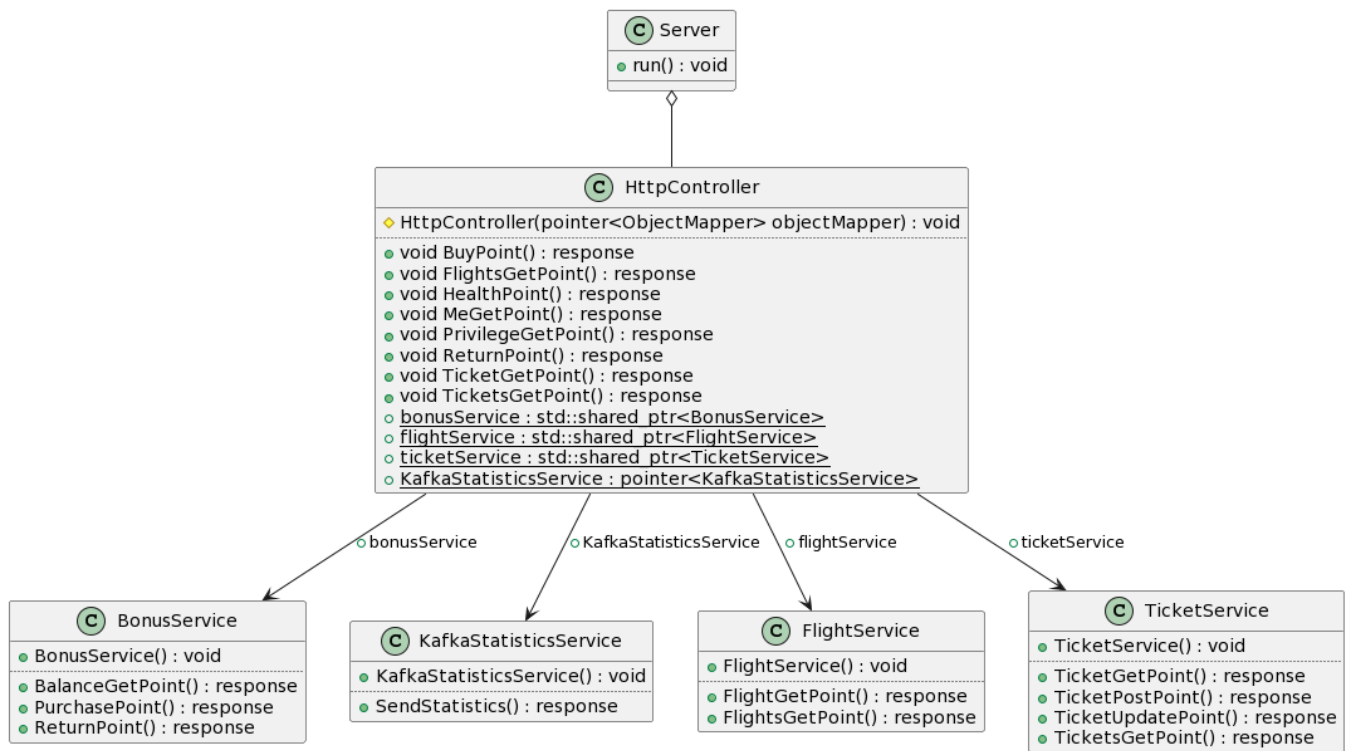


Рисунок 10 – Диаграмма классов сервиса-координатора

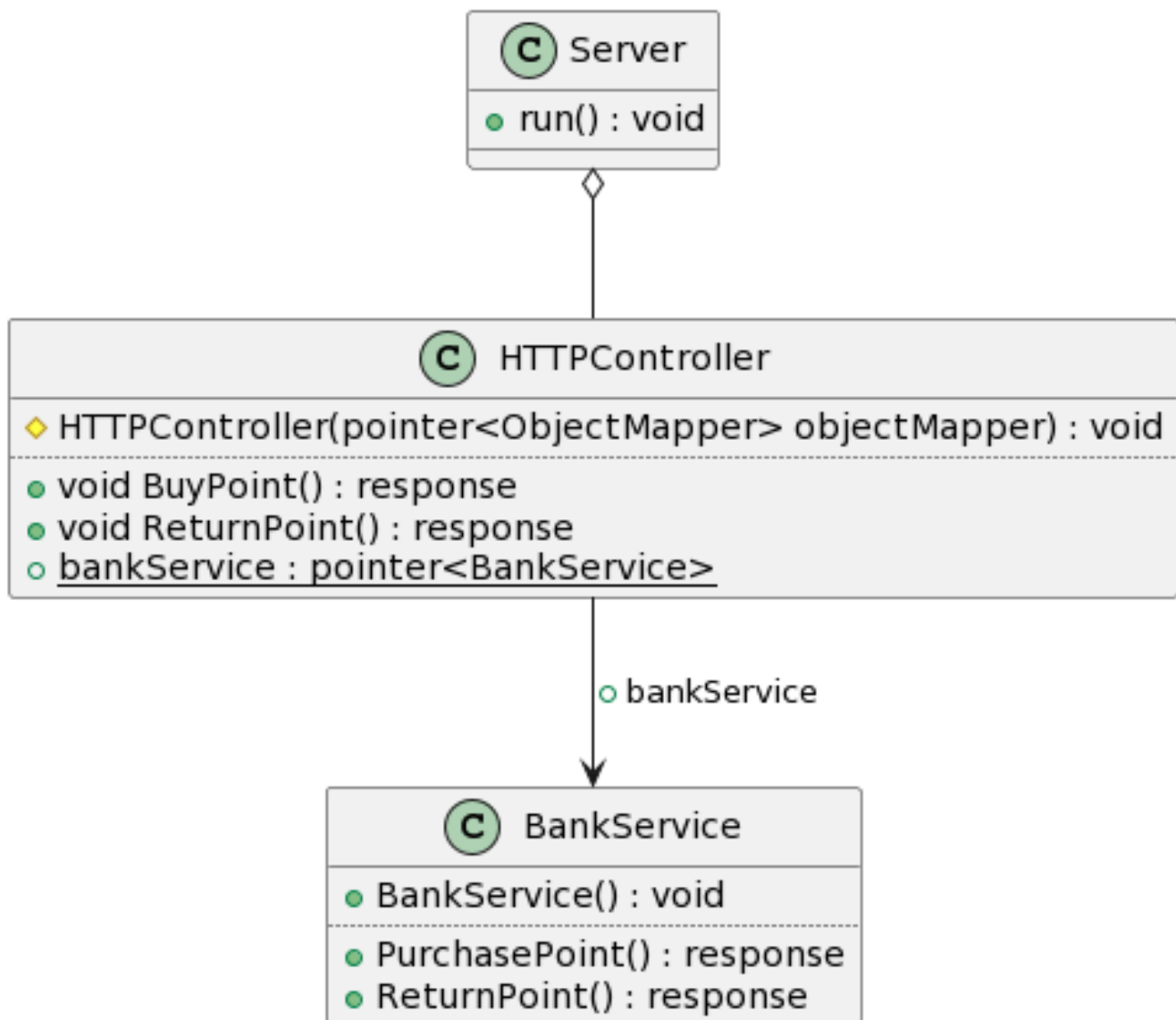


Рисунок 11 – Диаграмма классов сервиса оплаты

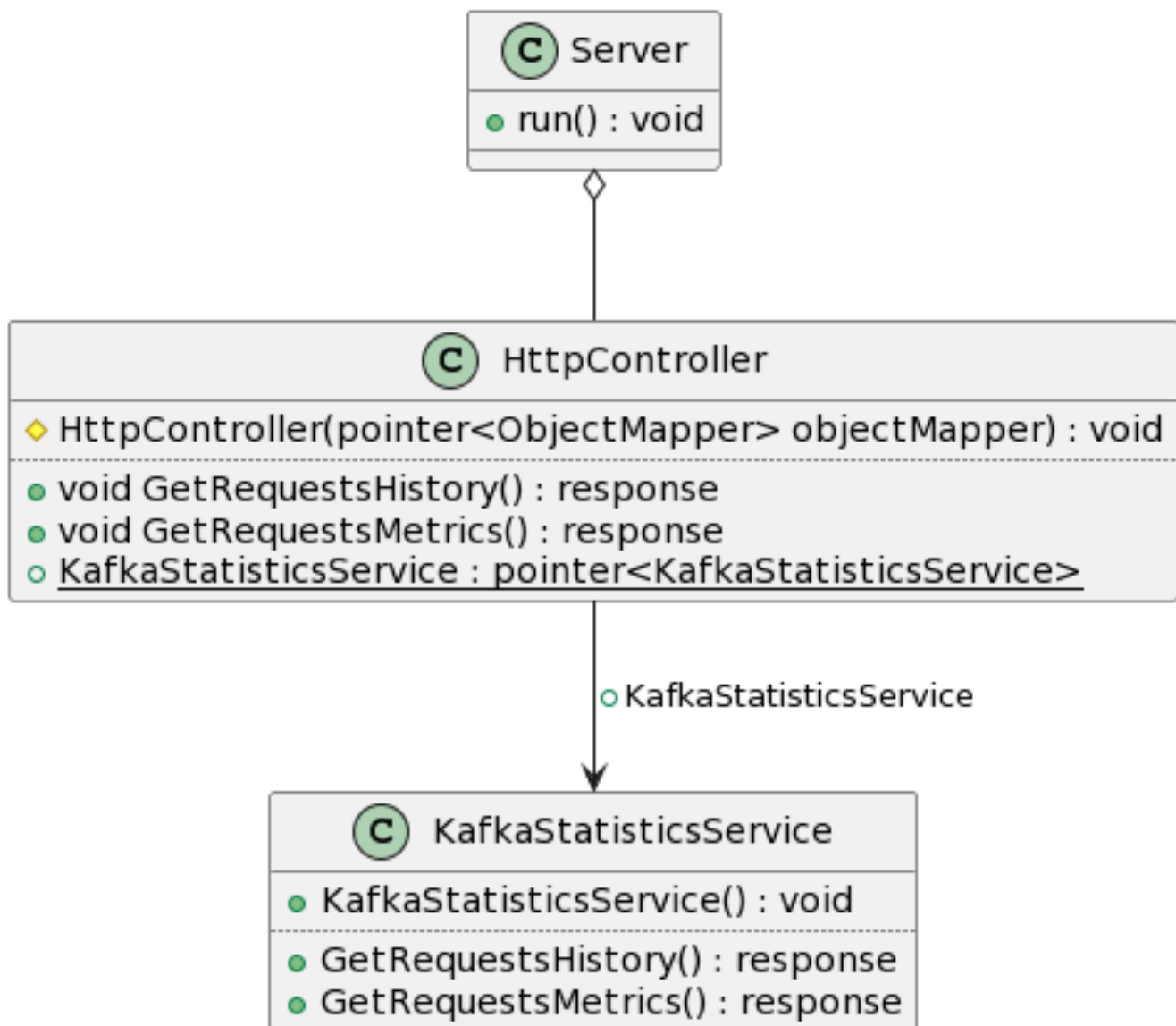


Рисунок 12 – Диаграмма классов сервиса статистики

Описание классов сервисов

Сервисы рейсов, билетов, программы лояльности и сессий и пользователей спроектированы похожим образом. Они имеют:

- слой доступа к данным, реализованный с помощью паттерна «Репозиторий» для абстракции хранения, а также паттерна «пул объектов» для эффективного использования активных подключений к базе данных;
- слой логики работы сервиса;
- слой интерфейса – слой связи с другими сервисами с помощью http-запросов.

Так как сервисы спроектированы одинаково, часть классов совпадают. Опишем их:

- IDAFacade – абстрактный интерфейс слоя доступа к данным; PGDAFacade – реализация этого абстрактного интерфейса для работы с данными, хранящимися под управлением СУБД Postgres;
- IDAFactory – абстрактный интерфейс фабрики объектов, относящихся к библиотеке работы с данными (интерфейс спроектирован в соответствии с паттерном «фабрика объектов»); PGDAFactory – реализация этого абстрактного интерфейса для работы с данными, хранящимися под управлением СУБД Postgres;
- PGConnection – пул активных подключений к базе данных, спроектированный в соответствии с паттерном «пул объектов»;
- IBLFacade – абстрактный интерфейс слоя логики; BLFacade – реализация этого абстрактного интерфейса;
- IServer – абстрактный интерфейс серверного приложения, предоставляющего интерфейс, описанный HTTPController; Server – реализация этого интерфейса.
- HTTPController – класс, описывающий набор HTTP-методов, которые доступны на сервисе. Фактически занимается распаковкой данных, пришедших по сети, заполнением необходимых структур этими данными и

вызовом функций, реализующих логику работы методов. Также занимается подготовкой результирующих данных к отправке по сети после выполнения запроса.

Кроме того, каждый сервис имеет собственные классы. Опишем их.

Описание классов сервиса рейсов

- `Flight` – класс, описывающий рейс, имеет поля:
 - `datetime : string` - дата и время рейса;
 - `price : int` - цена рейса;
 - `flight_number : int` - номер рейса;
 - `from_airport_name/to_airport_name : string` - аэропорты вылета и прилёта;
 - `from_airport_city/to_airport_city : string` - города вылета и прилёта.
- `IFlightRepository` – абстрактный интерфейс для работы с данными рейсов (интерфейс спроектирован в соответствии с паттерном «репозиторий»).
- `PGFlightRepository` – реализация этого абстрактного интерфейса для работы с данными, хранящимися под управлением СУБД Postgres.

Описание классов сервиса билетов

- `Ticket` – класс, описывающий билет, имеет поля:
 - `id : int` - номер билета;
 - `price : int` - цена билета;
 - `flight_number : string` - номер рейса;
 - `status : bool` - статус (оплачен или отменён);
 - `ticket_uid : string` - идентификатор билета;
 - `username : string` - владелец билета.

- ITicketRepository – абстрактный интерфейс для работы с данными билетов (интерфейс спроектирован в соответствии с паттерном «репозиторий»). PGTicketRepository – реализация этого абстрактного интерфейса для работы с данными, хранящимися под управлением СУБД Postgres.

Описание классов сервиса сессий и пользователей

- User – класс, описывающий пользователя, имеет поля:
 - id : int - идентификатор пользователя;
 - login : string - логин пользователя;
 - role : string - роль пользователя;
 - age : int - возраст пользователя;
 - mobilePhone : string - номер телефона
 - email : string - почта
 - password : string - пароль пользователя (в виде хэша).
- Session – класс, описывающий сессию пользователя, является JWT-token-ом;
- Credentials – класс, который хранит логин и пароль пользователя.
- IUserRepository – абстрактный интерфейс для работы с данными пользователей и сессий (интерфейс спроектирован в соответствии с паттерном «репозиторий»). PGUserRepository – реализация этого абстрактного интерфейса для работы с данными, хранящимися под управлением СУБД Postgres.
- BuyRequest – класс, описывающий запрос на покупку билета, имеет поля:
 - решение пользователя о списании или о начислении бонусов;
 - цена билета;
 - номер билета;
 - имя пользователя.

- `BuyResponse` – класс, описывающий ответ на покупку билета, имеет поля:
 - баланс счёта программы лояльности после покупки билета;
 - количество денег, списанное за счёт бонусов;
 - количество денег, списанное за счёт средств пользователя;
 - статус счёта лояльности (золотой, серебряный, бронзовый) после покупки.
- `BalanceResponse` – класс, описывающий ответ на запрос информации о бонусном счёте и истории операций бонусного счёта, имеет поля:
 - баланс счёта программы лояльности;
 - статус счёта лояльности (золотой, серебряный, бронзовый);
 - история операций (каждая запись содержит: абсолютное значение изменения баланса, дату операции, типа операции – списание или начисление, идентификатор билета).
- `IBonusRepository` – абстрактный интерфейс для работы с данными программы лояльности (интерфейс спроектирован в соответствии с паттерном «репозиторий»). `PGBonusRepository` – реализация этого абстрактного интерфейса для работы с данными, хранящимися под управлением СУБД Postgres.

Описание классов сервиса-координатора

- `HTTPController` – класс, описывающий набор HTTP-методов, которые доступны на сервисе. Фактически предасвляет собой весь программный интерфейс системы. В своей работе для обслуживания приходящих запросов сервис использует интерфейсы сервисов программы лояльности (`BonusService`), рейсов (`FlightService`) и билетов (`TicketService`). Кроме того, сервис перенаправляет статистику запросов в очередь Kafka с помощью интерфейса `KafkaStatisticsService`.
- `Server` – реализация серверного приложения, предоставляющего интерфейс, описанный `HTTPController`.

Описание классов сервиса оплаты

- HTTPController – класс, описывающий набор HTTP-методов, которые доступны на сервисе (покупка и возврат билетов). В своей работе для обслуживания приходящих запросов сервис использует сервисы банка для выполнения транзакций со счётом пользователя.
- Server – реализация серверного приложения, предоставляющего интерфейс, описанный HTTPController.

Описание классов сервиса статистики

- HTTPController – класс, описывающий набор HTTP-методов, которые доступны на сервисе (просмотр истории запросов к системе и метрики, рассчитанные на основе этой истории). В своей работе для обслуживания приходящих запросов сервис использует очередь Kafka.
- Server – реализация серверного приложения, предоставляющего интерфейс, описанный HTTPController.

Диаграмма действий

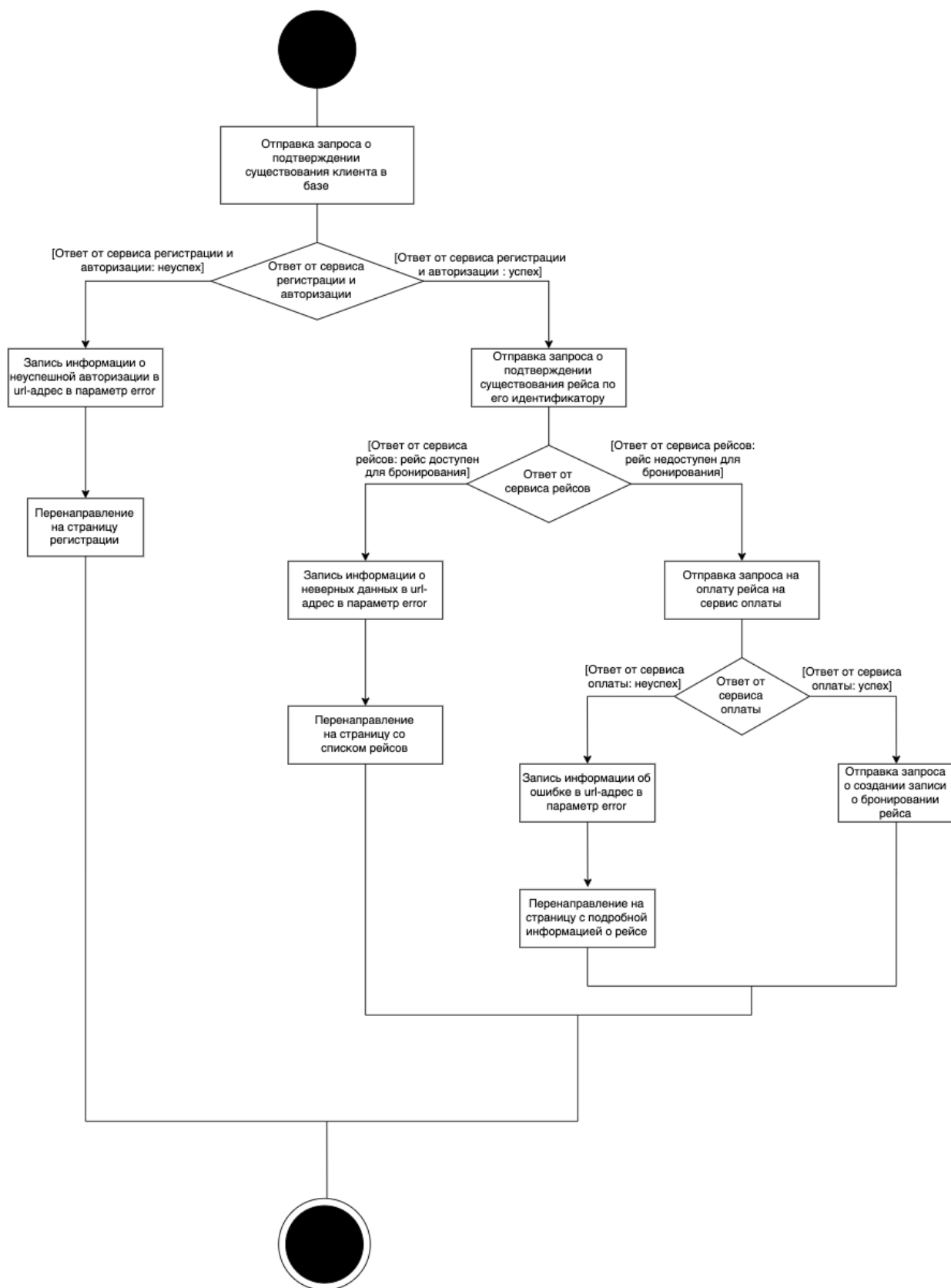


Рисунок 13 – Страница со статистикой

Диаграмма последовательности действий

Диаграмма последовательности действий представлена на рисунке 14.

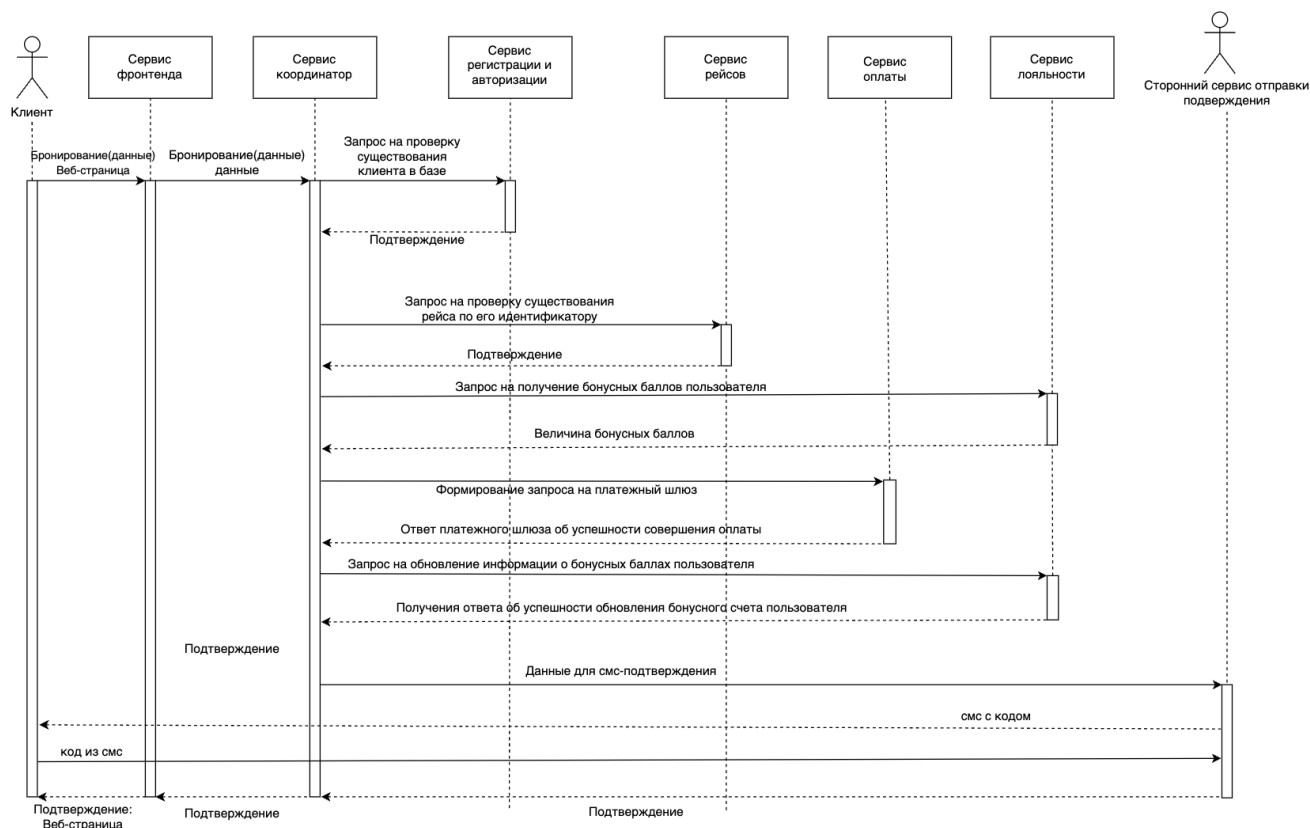


Рисунок 14 – Диаграмма последовательности действий

Диаграмма потоков данных

Рассматриваемая система предполагает распределенное хранение данных. Диаграмма потоков данных, представленная на рисунке 15, позволяет описать распределение сохраняемой информации в хранилищах данных.

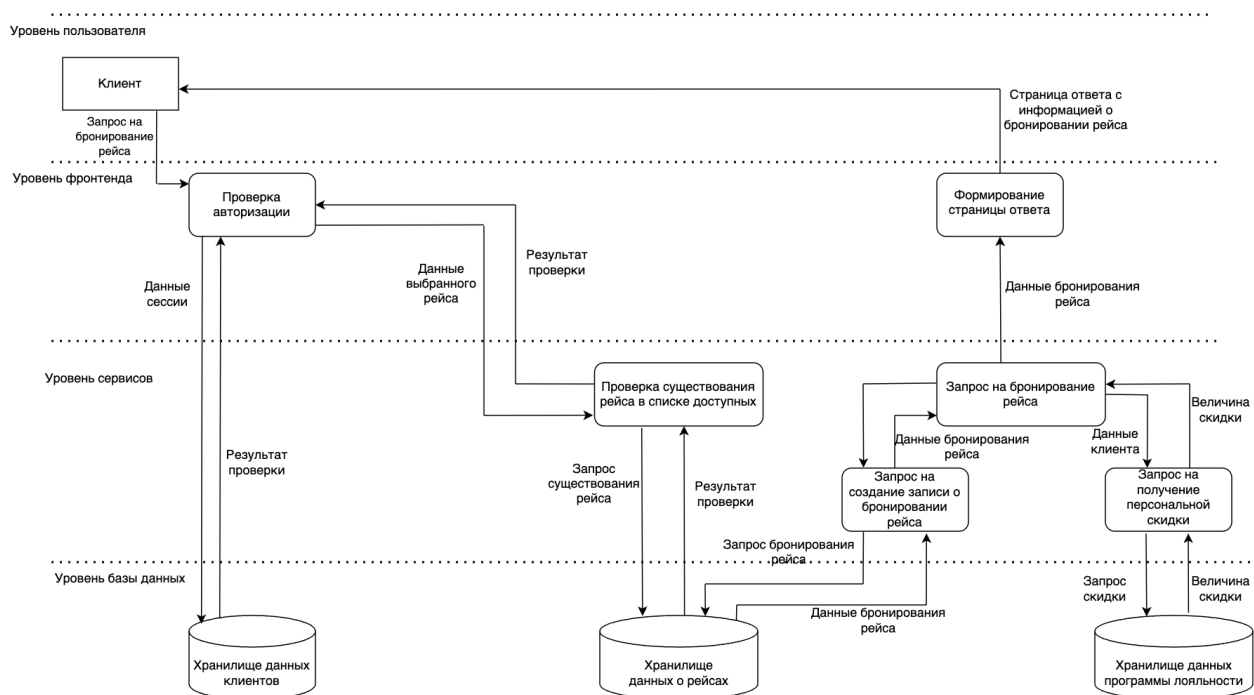


Рисунок 15 – Диаграмма потоков данных

Высокоуровневый дизайн пользовательского интерфейса

Пользовательский интерфейс в разрабатываемой системе представляет собой web-интерфейс, доступ к которому осуществляется через браузер (тонкий клиент). Страница системы состоит из «шапки», бокового меню и основной части.

Обобщенно структуру страниц системы можно представить следующим образом:

- страница авторизации;
- страница регистрации;
- страница со списком полетов;
- страница со списком купленных и сданных билетов;
- страница с информацией о пользователе и историей покупок билетов;
- страница со статистикой сервиса-координатора.

Данные страницы представлены на рисунках приведены примеры описанных страниц.

The image shows a web browser window displaying the login page of a system named 'SkyReserve'. The header is blue and contains the 'SkyReserve' logo on the left and two buttons, 'Авторизоваться' (Login) and 'Зарегистрироваться' (Register), on the right. The main body of the page is light gray. In the center, there is a white rectangular box titled 'Авторизация' (Login). Inside this box, there are two input fields: the first is labeled 'Логин*' (Login*) and has an eye icon to its left; the second is labeled 'Пароль*' (Password*) and has a lock icon to its left. Below these two fields is a solid blue button with the white text 'Войти' (Login).

Рисунок 16 – Страница авторизации

☰ SkyReserve Авторизоваться Зарегистрироваться

✈

Регистрация

Логин*

Пароль*

Фамилия*

Имя*

Почта*

Телефон*

Зарегистрироваться

Рисунок 17 – Страница регистрации

☰ SkyReserve Выйти

✈

Номер	Отправление	Прибытие	Время	Цена	
AFL031	Санкт-Петербург Пулково	Москва Шереметьево	2021-10-08 20:00	1500	
SU1226	Москва Д			4800	
S72046	Сочи Ад			3900	
SU6190	Нижний			5600	
S72051	Санкт-Пе			6700	
SU6039	Москва Д			8500	
S72053	Москва Л			2500	
U6-256	Москва Л			3700	
SU1890	Сочи Ад			7400	
U6-2803	Нижний			3300	
AFL032	Санкт-Пе			1500	
SU1227	Москва Д			4800	
S72047	Сочи Адлер	Москва Шереметьево	2024-08-06 20:00	3900	
SU6191	Нижний Новгород Стригино	Москва Домодедово	2024-08-15 06:00	5600	
S72052	Санкт-Петербург Пулково	Нижний Новгород Стригино	2024-08-20 20:00	6700	

Строк на странице: 20 1-20 из 25

Поиск полетов

Номер полета

Аэропорт отправления

Аэропорт прибытия

Время вылета

Время прилета

Минимальная цена

Максимальная цена

Найти Закрыть

Рисунок 18 – Страница с полетами

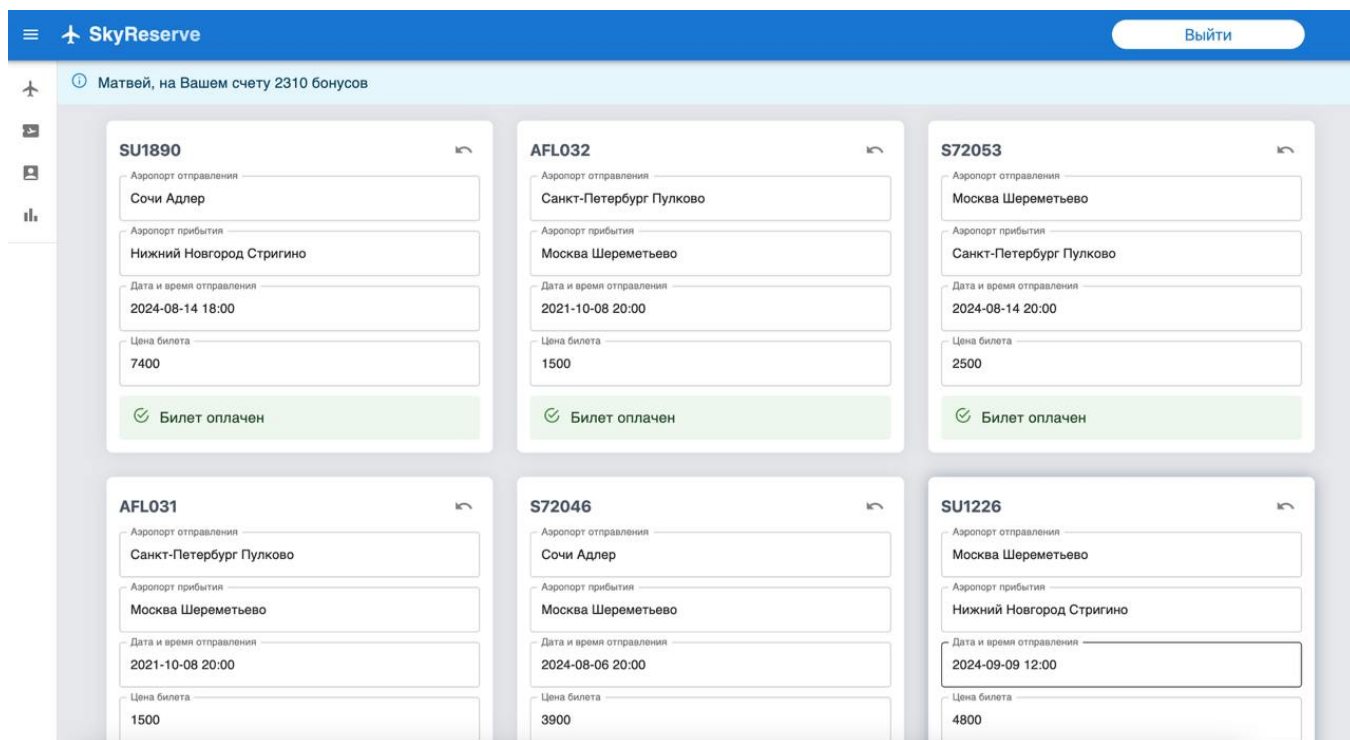


Рисунок 19 – Страница с купленными и сданными билетами

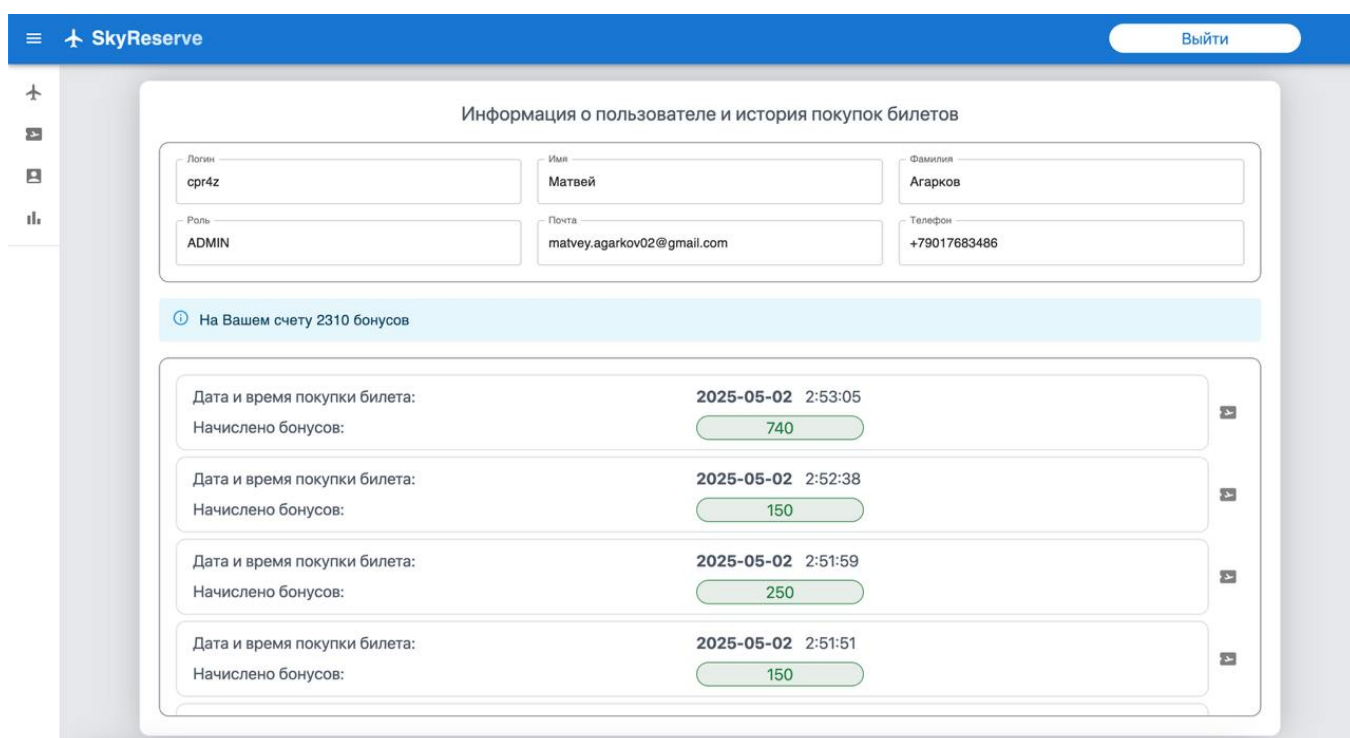


Рисунок 20 – Страница с информацией о пользователе и историей покупок

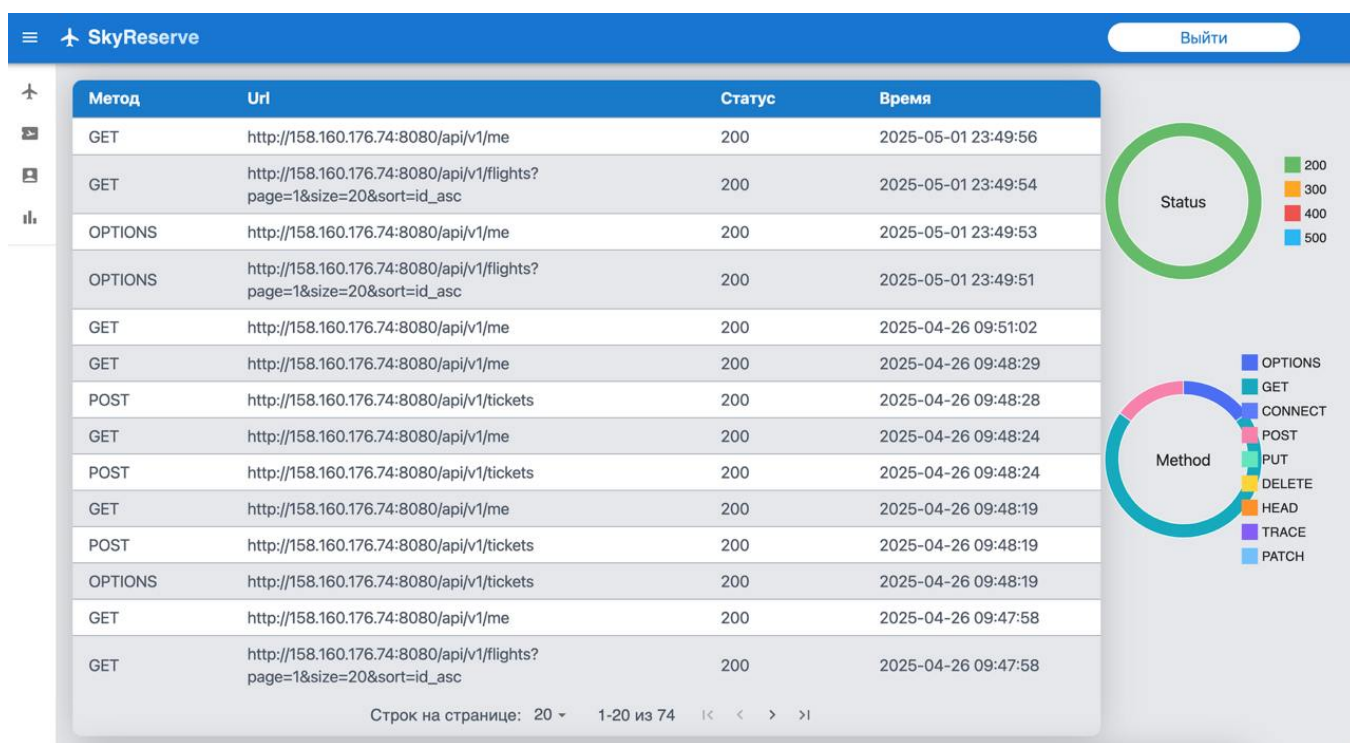


Рисунок 21 – Страница со статистикой

Физический дизайн

Выбор системы развертывания компонентов распределенной системы

Согласно требованиям технического задания, разрабатываемая система должна быть распределенной. Характерной особенностью распределенных систем является высокое многообразие используемых технологий. Особенно непростая ситуация возникает, когда разные компоненты системы используют разные версии одной и той же библиотеки. Для того чтобы компоненты не конфликтовали друг с другом, необходимо ввести требование изолированности. В этом случае приходится использовать отдельные серверы, что может быть экономически нецелесообразно, либо использовать контейнеризацию.

Контейнеризация – это методология разработки и управления приложениями, которая позволяет упаковывать приложение и все его зависимости в изолированный контейнер (образ изолируемой части системы, содержащий приложение со всеми его зависимостями). Контейнеры обеспечивают среду выполнения для приложения, которая полностью отделена от других контейнеров и основной системы. Это облегчает развертывание, масштабирование и управление приложениями, а также обеспечивает надежность и безопасность

при работе в различных средах.

В качестве платформы для автоматизации развертывания, масштабирования и управления контейнеризированными приложениями на серверах было решено использовать Kubernetes. Такое решение было принято в результате следующего сравнительного анализа.

Существует несколько аналогов Kubernetes, которые также предоставляют возможности для управления контейнеризированными приложениями:

1. Docker Swarm – это оркестратор контейнеров, разработанный Docker, который позволяет управлять кластером Docker-хостов и запускать контейнеры в них. Он более прост в использовании по сравнению с Kubernetes, но может быть менее мощным в некоторых аспектах.
2. Apache Mesos – это распределенная система управления ресурсами, которая также поддерживает запуск контейнеров. Он предоставляет более общий подход к управлению ресурсами и приложениями, чем Kubernetes.
3. Amazon ECS (Elastic Container Service) – это управляемый сервис от Amazon Web Services для запуска и управления контейнерами на инфраструктуре AWS. Он предлагает простой способ запуска и масштабирования контейнеров без необходимости управления инфраструктурой.

Достоинства Kubernetes по сравнению с аналогами:

- Мощные возможности оркестрации: Kubernetes предоставляет широкий спектр возможностей для управления и автоматизации развертывания приложений в контейнерах.
- Большое сообщество и экосистема: Kubernetes имеет активное сообщество разработчиков и широкий выбор инструментов и плагинов для расширения его функциональности.
- Поддержка различных облачных и локальных сред: Kubernetes поддерживает различные облачные провайдеры и может быть развернут как локально, так и в облаке.

Выбор операционной системы

Согласно требованиям технического задания, разрабатываемый портал должен обладать высокой доступностью, работать на типичных архитектурах ЭВМ (Intel x86, Intel x64), а так же быть экономически недорогим для сопровождения. Таким образом, можно сформулировать следующие требования к операционной системе:

- Распространенность. На рынке труда должно быть много специалистов, способных администрировать распределенную систему, работающую под управлением выбранной операционной системы.
- Надежность. Операционная система должна широко использоваться в стабильных проектах, таких как Mail.Ru, Vk.com, Google.com. Эти компании обеспечивают высокую работоспособность своих сервисов, и на их опыт можно положиться.
- Наличие требуемого программного обеспечения. Выбор операционной системы не должен ограничивать разработчиков в выборе программного обеспечения, библиотек.
- Цена.

Под данные требования лучше всего подходит ОС Ubuntu. Ubuntu – это дистрибутив, использующий ядро Linux. Как и все дистрибутивы Linux, Ubuntu является ОС с открытым исходным кодом, бесплатным для использования.

Выбор СУБД

В качестве СУБД была выбрана PostgreSQL, так как она наилучшим образом подходит под требования разрабатываемой системы:

- Масштабируемость: PostgreSQL поддерживает горизонтальное масштабирование, что позволяет распределить данные и запросы между несколькими узлами базы данных. Это особенно полезно в географически распределенных системах, где данные и пользователи могут быть разбросаны по разным регионам.

- Географическая репликация: PostgreSQL предоставляет возможность настройки репликации данных между различными узлами базы данных, расположенными в разных географических зонах. Это позволяет обеспечить отказоустойчивость и более быстрый доступ к данным для пользователей из разных частей мира.
- Гибкость и функциональность: PostgreSQL обладает широким набором функций и возможностей, что делает его подходящим для различных типов приложений и использования в распределенной среде. Он поддерживает сложные запросы, транзакции, хранимые процедуры и многое другое.
- Надежность и отказоустойчивость: PostgreSQL известен своей надежностью и стабильностью работы. В распределенной географической системе это особенно важно, поскольку он способен обеспечить сохранность данных и доступность даже при сбоях в отдельных узлах.

Выбор языка разработки и фреймворков компонент портала

Проанализируем техническое задание на разработку портала. Исходя из приведенных требований к системе, можно выявить требования к языку программирования:

- Совместимость с выбранными ранее технологиями. Выбранный язык должен уметь взаимодействовать с ОС Linux, СУБД PostgreSQL.
- Производительность: C++ является компилируемым языком программирования, что позволяет создавать быстродействующие приложения. Он обладает низким уровнем абстракции, что позволяет разработчику более тонко управлять ресурсами и оптимизировать производительность программы.
- Расширяемость: C++ обладает возможностью использовать объектно-ориентированный подход к программированию, что делает его удобным для создания сложных и масштабируемых систем.

Выбор oatpp в качестве фреймворка для разработки программного обеспечения также имеет свои преимущества и может быть обоснован следующими аспектами:

- Высокая производительность: oatpp является легковесным и быстрым фреймворком, который спроектирован для обеспечения высокой производительности приложений. Он оптимизирован для работы с сетевыми запросами и обработки данных, что позволяет создавать эффективные веб-сервисы.
- Поддержка многопоточности: oatpp предоставляет удобные инструменты для работы с многопоточностью, что позволяет создавать параллельные и распределенные приложения. Это особенно важно для систем, где требуется обработка большого количества запросов одновременно.
- RESTful API: oatpp поддерживает разработку RESTful API, что делает его удобным выбором для создания веб-сервисов и API. Он предоставляет инструменты для удобной маршрутизации запросов, валидации данных и других задач, связанных с разработкой веб-приложений.
- Модульность и расширяемость: oatpp построен на основе модульной архитектуры, что позволяет легко добавлять новый функционал и расширять возможности фреймворка. Это делает его гибким инструментом для разработки различных типов приложений.

Обеспечение надежности портала

В данной системе для обеспечения надежности функционирования СУБД будет применяться репликация и шардинг. Для обеспечения надежности данных СУБД необходимо разработать скрипт для автоматического создания резервной копий базы данных по расписанию.

Для фронтенда и бекендов целесообразно применить зеркалирование. Это обеспечит отказоустойчивость системы: в случае сбоя любого из ее узлов запросы на чтение данных будут выполняться.